14 haftalık yol haritasında 4. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 04

Arama Algoritmaları

Doğrusal Arama, İkili Arama ve Varyantları

MODÜL II · ARAMA VE SIRALAMA · H4–H7

> **HAFTANIN BÜYÜK SORUSU**
>
> *Bir milyon isimlik sıralı bir listede herhangi bir ismi en fazla kaç soruda bulabilirsiniz ve neredeyse tüm ikili arama gerçekleştirimleri neden yıllarca hatalı kaldı?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları.... 4
1.1 Öğrenme çıktıları.... 4
2. Geçen Haftadan ve AP-1'den Bağlantı.... 5
3. Kavramsal Anlatım: Aday Kümesini Yarıya İndirmek.... 5
3.1 Arama problemi ve sözleşmesi.... 5
3.2 Azalt ve fethet: her soruda adayların yarısı.... 5
3.3 Aralık sözleşmesi: kapalı [lo, hi] ve yarı açık [lo, hi).... 6
3.4 Monoton yüklem: “ilk doğru”yu aramak.... 8
3.5 Kaputun Altında.... 8
4. Algoritmalar.... 9
4.1 Doğrusal arama ve nöbetçi.... 9
4.2 İkili arama.... 11
4.3 lower_bound ve upper_bound: tekrar eden anahtarlar.... 15
4.4 Cevap üzerinde ikili arama.... 18
4.5 Gerçel sayılarda ikili arama: ikiye bölme yöntemi.... 20
4.6 Merdiven araması: satır ve sütunları sıralı matris.... 21
5. Çözümlü Örnekler.... 24
5.1 Orta nokta taşması: int32 sınırında (lo + hi) / 2 (Örnek 6, Δ).... 24
5.2 Ekleme konumu ve sıralı diziye ekleme (Örnek 8).... 25
5.3 Döndürülmüş sıralı dizide arama (Örnek 9).... 26
5.4 Genel arama: karşılaştırıcı işaretçisi ve bsearch (Örnek 12).... 28
5.5 Doğrusal ve ikili arama: sayaç ve süre (Örnek 13).... 29
5.6 Kısa bakış: üstel arama ve interpolasyon araması.... 29
6. Haftanın Deneyi: Doğrusal ve İkili Arama, Kesişim Noktası.... 30
7. Hata Avcısı.... 32
8. Dil Penceresi: C · C++ · Python.... 34
9. Mühendis Gözüyle.... 36
10. Mini-OBS 2 v2.3: Numarayla İkili Arama ve Soyad Öneki.... 37
11. Alıştırmalar ve Atölye.... 40
11.1 İzleme ve çıktı tahmini (A).... 40
11.2 Karmaşıklık (B).... 40
11.3 Değişmez yazma (C).... 41
11.4 Hata bulma (D).... 41
11.5 Parsons (E).... 42

11.6 Kod yazma (F)..... 42
11.7 Diller arası okuma (G)..... 43
11.8 Atölye 1 · “Doğru ikili aramayı yaz” meydan okuması (bireysel, ≈ 30 dk)..... 43
11.9 Atölye 2 · Sayı tahmin oyunu ve soru sayısı istatistiği (eşli, ≈ 20 dk)..... 45
11.10 Atölye 3 · Varyant avı: lower_bound ile beş problem (eşli, ≈ 25 dk)..... 45
12. Kendini Değerlendirme..... 46
13. İleri Bakış ve Merak Kutusu..... 46
14. Özet, Kavram Haritası ve Terim Sözlüğü..... 47
15. Kaynaklar..... 48
Ek A. Dil Karşılıkları..... 49
A.1 Örnek 1 · Doğrusal arama: ilk konum, bulunamazsa −1 / None; sıralı dizide erken çıkış..... 49
A.2 Örnek 2 · Nöbetçili doğrusal arama: sınır denetimi kalkar, karşılaştırma sayısı 2n + 1'den n + 2'ye iner..... 50
A.3 Örnek 3 · İkili arama — yinelemeli, kapalı aralık [lo, hi]; yoklama sayısı ≤ $\lfloor \log_2 n \rfloor + 1$..... 50
A.4 Örnek 4 · İkili arama — yarı açık aralık [lo, hi) ve değişmez assert'leri; ekleme konumu..... 51
A.5 Örnek 5 · İkili arama — özyinelemeli: T(n) = T(n/2) + c; çağrı ve derinlik sayaçları..... 52
A.6 Örnek 6 · Orta nokta taşması: (lo + hi) / 2 ile int32 sınırında hata ve güvenli lo + (hi − lo) / 2… 53
A.7 Örnek 7 · lower_bound ve upper_bound; ilk/son konum ve bir değerin kaç kez geçtiği..... 55
A.8 Örnek 8 · Ekleme konumu ve sıralı diziye ekleme: arama O(log n), kaydırma O(n)..... 56
A.9 Örnek 9 · Döndürülmüş sıralı dizide arama; döndürme noktası; tekrar eden elemanlarda O(n) 57
A.10 Örnek 10 · Cevap üzerinde ikili arama: tamsayı karekök ve sayı tahmin oyunu (en çok $\lfloor \log_2 N \rfloor + 1$ soru)..... 58
A.11 Örnek 11 · İkiye bölme yöntemiyle denklem kökü: $x^3 - x - 2 = 0$; tolerans ve yineleme sayısı 59
A.12 Örnek 12 · Genel arama: karşılaştırıcı işaretçisiyle struct dizisinde arama; bsearch / std::lower_bound / bisect..... 60
A.13 Örnek 13 · Doğrusal ve ikili arama süre karşılaştırması ve kesişim noktası..... 61
A.14 Örnek 14 · Satır ve sütunları sıralı matriste merdiven araması: en çok m + n − 1 adım..... 62

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir milyon isimlik sıralı bir listede herhangi bir ismi en fazla kaç soruda bulabilirsiniz ve neredeyse tüm ikili arama gerçekleştirimleri neden yıllarca hatalı kaldı?

Sözlükte bir sözcüğü ararken ortalardan bir yer açar, aradığımızın önde mi arkada mı kaldığına bakar ve kalan yarıda aynı şeyi yaparız. Her bakış adayların yarısını eler: bir milyon isimlik sıralı listede 1 000 000 → 500 000 → … → 1 ve yirminci bakışta tek isim kalır, çünkü $2^{20} = 1\,048\,576 > 1\,000\,000$. Örnek 3'te 10^6 elemanlı dizide 1000 rastgele sorgunun hiçbiri 20 yoklamadan fazlasını istemez; Örnek 10'un sayı tahmin oyununda $1..10^6$ arasındaki her sayı en çok 20 soruda, ortalama 18,95 soruda bulunur (tutulan 765 432, 20 soruda bulunur).

Sorunun ikinci yarısı daha şaşırtıcıdır: bu kadar kısa bir algoritmayı doğru yazmak neden zordur? İkili aramanın kırılgan olduğu dört yeri inceleyeceğiz: aralığın sınırları (hi = n mi, n − 1 mi?), döngü koşulu (< mı, ≤ mü?), daraltma adımı (mid mi, mid + 1 mi?) ve orta nokta hesabı. Sonra fikri dizilerin dışına taşıyacağız: karekök, denklem kökü, kamyon kapasitesi ve hatayı getiren ilk sürüm de “ilk doğru”yu arayan ikili aramadır.

> **GERÇEK DÜNYADAN · 2006: “Neredeyse bütün ikili aramalar hatalı”**
>
> Joshua Bloch, 2 Haziran 2006'da Google Research Blog'daki yazısında Java kütüphanesi (`java.util.Arrays`) için yazdığı `binarySearch`'teki `int mid = (low + high) / 2;` satırının hatalı olduğunu duyurdu: low + high toplamı $2^{31} - 1$'i aşınca negatife taşar, ikiye bölünce de negatif kalır. Java'da bu `ArrayIndexOutOfBoundsException` fırlatır; C'de öngörülemeyen sonuçlarla dizi dışı erişime yol açar. Bloch'a göre hata 2^{30} (yaklaşık bir milyar) ya da daha fazla elemanlı dizilerde ortaya çıkabilir (kesin sınır Bölüm 5.1'de). JDK'deki sürüm yaklaşık dokuz yıl fark edilmeden kalmış; Bentley'nin “Programming Pearls” kitabında doğruluğu kanıtlanıp sınanmış ikili aramanın karşılık gelen satırı (m'yi l ile u'nun aşağı yuvarlanmış ortalaması yapan satır) da aynı taşmayı taşıyordu ve yirmi yıl gözden kaçmıştı (Bloch, 2006).
>
> Bloch Java için iki düzeltme önerir: `int mid = low + ((high - low) / 2);` ve işaretsiz sağa kaydırmayla `(low + high) >>> 1`. İkincisi Java'da doğrudur, çünkü Java'da `int` taşması tanımlıdır (ikiye tümleyen sarma): toplam negatif görünse de bitleri doğru işaretsiz toplamdır ve `>>>` soldan 0 kaydırarak onu ikiye böler. C ve C++'ta işaretli taşma tanımsız olduğundan aynı fikir ancak işlenenler önce `unsigned`'a çevrilerek uygulanabilir; yazının 2008 güncellemesine göre C/C++ için önerilen ilk düzeltmenin C99'da güvencesi yoktu ve düzeltme değiştirildi.
>
> Sorun yeni de değildir: Bentley'nin Knuth'a dayanarak aktardığına göre ilk ikili arama 1946'da yayımlanmış, her n için doğru çalışan ilk sürüm 1962'ye kadar görülmemişti (Bentley, 2000; Bloch, 2006). Bentley, birkaç saat süre verdiği profesyonel programcıların yaklaşık yüzde doksanının programında hata çıktığını bildirir (Bentley, 2000). Aynı meydan okumayı ortak bir test paketiyle deneyeceğiz (Atölye 1).

1.1 Öğrenme çıktıları

- **ÖÇ 4.1** Doğrusal aramayı (nöbetçili sürüm dahil) gerçekler; en iyi, en kötü ve ortalama durum karmaşıklığını çıkarır. (Bölüm 4.1, Örnek 1–2)
- **ÖÇ 4.2** İkili aramayı yarı açık aralık değişmeziyle yinelemeli ve özyinelemeli olarak doğru biçimde gerçekler; O(log n) karmaşıklığını gerekçelendirir. (Bölüm 3.3, 4.2, Örnek 3–6)
- **ÖÇ 4.3** İkili arama varyantlarını (ilk/son konum, lower_bound/upper_bound, ekleme konumu, döndürülmüş dizide arama) uygular. (Bölüm 4.3, 5.2–5.3, Örnek 7–9)
- **ÖÇ 4.4** Monoton bir yüklem üzerinde “cevap üzerinde ikili arama” ile optimizasyon problemlerini çözer. (Bölüm 3.4, 4.4, Örnek 10–11)
- **ÖÇ 4.5** Fonksiyon işaretçileri ve karşılaştırıcılarla genel (generic) arama fonksiyonu yazar; bsearch, std::lower_bound ve bisect karşılıklarını kullanır. (Bölüm 5.4, 8, Örnek 12)

2. Geçen Haftadan ve AP-1'den Bağlantı

H1'de doğrusal aramayı, "bulunamadı" sözleşmesini (C'de −1, Python'da `None`) ve döngü değişmezinin (loop invariant) üç adımını yazdık: başlatma, koruma, sonlanma. **H2**'de ikiye bölünen döngünün $\lfloor \log_2 n \rfloor + 1$ adım sürdüğünü ve doğrusal aramanın başarılı aramada ortalama $(n + 1)/2$ karşılaştırma yaptığını ölçtük. **H3**'te yineleme bağıntılarını yerine koyma yöntemiyle çözdük ve özyineleme derinliğinin yığın belleğini belirlediğini gördük. İkili arama bu üç aracı birlikte kullanır: değişmezle doğruluk, $\log_2 n$ ile analiz, $T(n) = T(n/2) + c$ ile özyinelemeli sürüm.

AP-1'den iki kavram dönüyor: **H10**'daki adres formülü taban + i × sizeof(eleman), `bsearch`'ün her türde arama yapmak için bayt düzeyinde yaptığı hesaptır (Bölüm 3.5, 5.4); **H12**'nin merak kutusundaki fonksiyon işaretçisi ise karşılaştırma ölçütünü arama fonksiyonuna değer olarak vermemizi sağlar.

> TAHMİN ET · **Geri çağırma: üç soru (yanıtlar aşağıda)**
>
> - **S1 (H2).** n = 100'den başlayıp her adımda `n = n / 2` yapan döngü n = 0 olana kadar kaç adım sürer?
> - **S2 (H3).** $T(n) = T(n - 1) + c$ bağıntısının çözümü nedir? $T(n) = T(n/2) + c$ için yerine koyma yöntemi ne verir?
> - **S3 (H1).** Doğrusal aramanın döngü değişmezi "x, a[0..i − 1] içinde yok" ise döngü i = n ile bittiğinde ne söyleyebiliriz?

Yanıtlar. S1: 7 adım: 100 → 50 → 25 → 12 → 6 → 3 → 1 → 0 (H2, Örnek 3), yani $\lfloor \log_2 100 \rfloor + 1$. S2: $\Theta(n)$; $T(n/2) + c \to T(n/2^k) + k \cdot c$, $k = \log_2 n$: $\Theta(\log n)$. S3: değişmez bütün diziyi kapsar, x yoktur, −1 doğrudur. H2'nin ölçümü bu haftanın çıkış noktasıdır: n = 10'da başarılı aramaların ortalaması 5,50 = $(n + 1)/2$ karşılaştırmaydı.

3. Kavramsal Anlatım: Aday Kümesini Yarıya İndirmek

3.1 Arama problemi ve sözleşmesi

Arama problemi (search problem): a[0..n − 1] ve x verildiğinde a[k] = x olan bir k'yi ya da "bulunamadı" bilgisini döndür. Bu tanım bir **sözleşmedir** (contract) ve üç soruyu açıkça yanıtlamalıdır. (1) **Bulunamadı nasıl bildirilir?** C ve C++ örneklerimiz geçersiz indis −1'i, Python `None`'ı döndürür; `list.index` ise `ValueError` fırlatır (Hata Avcısı 6). (2) **Tekrar eden değerlerde hangi konum?** Doğrusal arama ilk konumu, kapalı aralıklı ikili arama eşitlerden **herhangi birini**, lower_bound **ilk** konumu verir; `bsearch` hangisini döndüreceğini belirtmez. (3) **Ön koşul ne?** İkili arama artan sıralı dizi ister; koşul bozulursa uyarı vermeden yanlış sonuç üretir (Hata Avcısı 4).

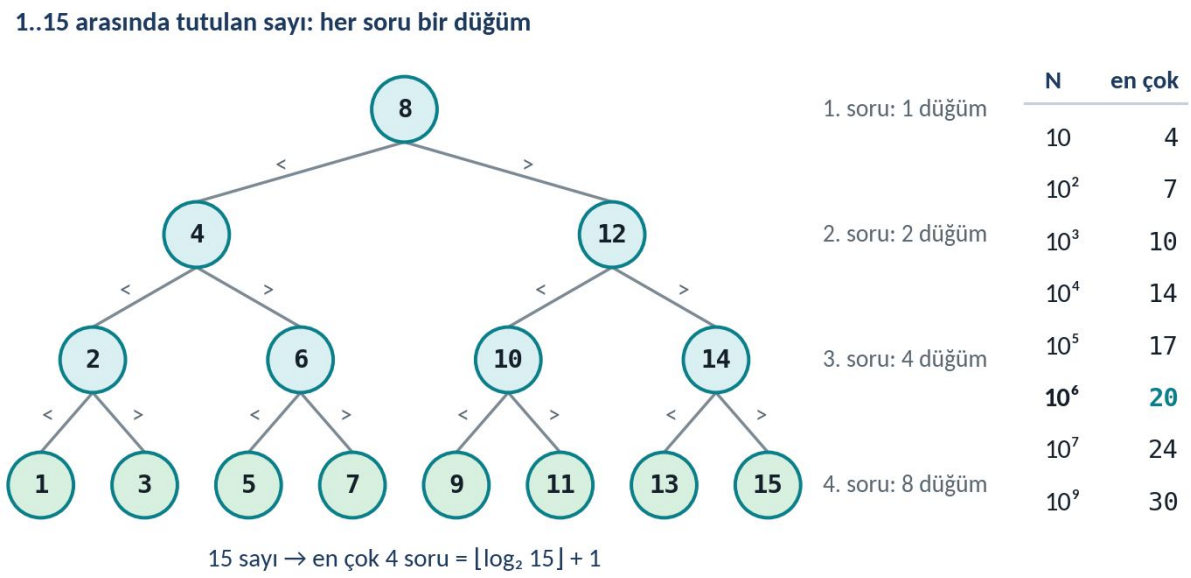
Sıralama bir **ön işlemedir** (preprocessing): bedeli bir kez ödenir, sonra her sorgu ucuzlar. Bu ödünleşimi (trade-off) Bölüm 9'da sayaçla ölçeceğiz: 100 000 sırasız anahtarda, bu girdide, sıralama 20 sorgudan sonra kâra geçer. Veri sık değişiyorsa sıralı tutmanın da bedeli vardır: her ekleme ortalama n/2 elemanı kaydırır (Örnek 8).

3.2 Azalt ve fethet: her soruda adayların yarısı

İkili arama bir **azalt ve fethet** (decrease and conquer) algoritmasıdır: problemi her adımda sabit oranda küçültür. Ortadaki a[mid] ile x'in karşılaştırması üç yanıttan birini verir: eşit (bulundu), a[mid] < x (x varsa sağ yarıda), a[mid] > x (x varsa sol yarıda). İncelenen her a[mid]'e **yoklama** (probe) diyeceğiz. m adaydan bir yoklamada en çok $\lfloor m/2 \rfloor$ aday kalır; n adaydan başlayan arama en çok $\lfloor \log_2 n \rfloor + 1$ yoklamada biter.

Bu sınır **her karşılaştırmalı aramanın** alt sınırıdır. Sorusu "x bu mu, değilse küçük mü, büyük mü?" olan bir yöntemi **karar ağacı** (decision tree) olarak çizelim (Şekil 4.1): her düğüm bir soru, sol/sağ çocuk "küçük"/"büyük" yanıtından sonraki sorudur. k düzeyli ağaçta en çok $2^k - 1$ düğüm, yani $2^k - 1$ farklı "bulundu" yanıtı vardır. n konumu ayırt etmek için $2^k - 1 \geq n$ gerekir; en küçük $k = \lceil \log_2(n+1) \rceil = \lfloor \log_2 n \rfloor + 1$'dir. İkili arama bu sınıra ulaşır: $n = 10^6$ için 20, $n = 10^9$ için 30 soru.

Şekil 4.1
1..15 için sayı tahmin oyununun karar ağacı (her düğüm bir soru, k. düzeyde 2^{k-1} düğüm) ve N'ye göre en çok soru sayısı $\lfloor \log_2 N \rfloor + 1$ (Örnek 10'un çıktısıyla aynı)



Doğrusal aramanın karar ağacı bir zincirdir: her soru tek aday eler. İkili aramanın üstünlüğü **rastgele erişime** (random access) dayanır: a[mid]'e O(1)'de ulaşmak gerekir. Bağlı listede (H8) ya da dosya akışında ortadaki elemana ulaşmak O(n) sürer.

3.3 Aralık sözleşmesi: kapalı [lo, hi] ve yarı açık [lo, hi)

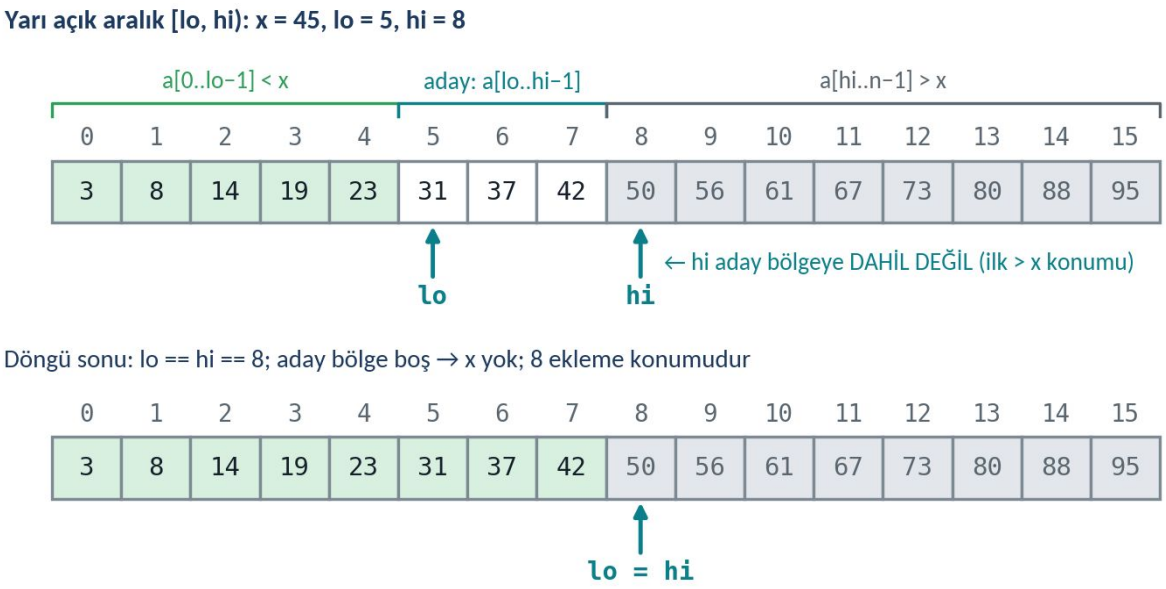
İkili aramanın inceliği aday bölgeyi iki indisle nasıl tarif ettiğimizdedir. İki yaygın sözleşmede **dört öge** birbirine uymalıdır: başlangıç, döngü koşulu ve iki güncelleme.

Tablo 4.1
İki aralık sözleşmesi: dört ögenin uyumu

Öge	Kapalı aralık [lo, hi]	Yarı açık aralık [lo, hi)
Aday bölge	`a[lo..hi]` (hi dahil)	`a[lo..hi − 1]` (hi hariç)
Başlangıç	`lo = 0, hi = n − 1`	`lo = 0, hi = n`
Aralık boş değilken	`lo <= hi`	`lo < hi`
a[mid] < x ise	`lo = mid + 1`	`lo = mid + 1`
a[mid] > x ise	`hi = mid − 1`	`hi = mid`
Döngü bitince	`lo == hi + 1` (aday yok)	`lo == hi` (aday yok); `lo` ekleme konumu
Boş dizi (n = 0)	`hi = −1`: işaretli indis gerekir	`hi = 0`: `size_t` ile güvenli

Sözleşme bir **değişmezdir**: “x dizideyse aday bölgededir”. Yarı açık sürümde daha güçlüsü yazılır: a[0..lo − 1] < x ve a[hi..n − 1] > x (Şekil 4.2). Başlangıçta iki dış bölge boştur. a[mid] < x ise a[lo..mid]'in hepsi < x'tir ve lo = mid + 1 değişmezi korur; a[mid] > x ise a[mid..hi − 1]'in hepsi > x'tir ve hi = mid korur. Döngü lo == hi ile bitince aday bölge boştur ve **lo ekleme konumudur**.

Şekil 4.2
Yarı açık aralık değişmezi: a[0..lo − 1] < x (yeşil), aday a[lo..hi − 1] (beyaz), a[hi..n − 1] > x (gri). Altta: döngü lo == hi ile bitti, lo ekleme konumu (Örnek 4, x = 45)



Sonlanma için aday sayısı (kapalıda hi − lo + 1, yarı açıkta hi − lo) her adımda **kesin** azalmalıdır. Kapalıda lo ≤ mid ≤ hi, yarı açıkta lo ≤ mid < hi olduğundan her sözleşmenin kendi güncellemeleri mid'i bölgeden çıkarır. Sözleşmeleri karıştırmak bu güvenceyi bozar: kapalı aralıkta `lo = mid` ya da `hi = mid` yazılırsa mid aday kalır ve aralık iki ya da bir adaya inince değişmez; döngü sonsuza dek sürer (Hata Avcısı 1). İki sözleşme de doğrudur ama farklı yoklama dizileri üretir: x = 61 için kapalı aralık 7, 11, 9, 10, yarı açık 8, 12, 10 numaralı elemanları yoklar (Şekil 4.3). Önemli olan birini seçip dört ögeyi ona göre yazmaktır.

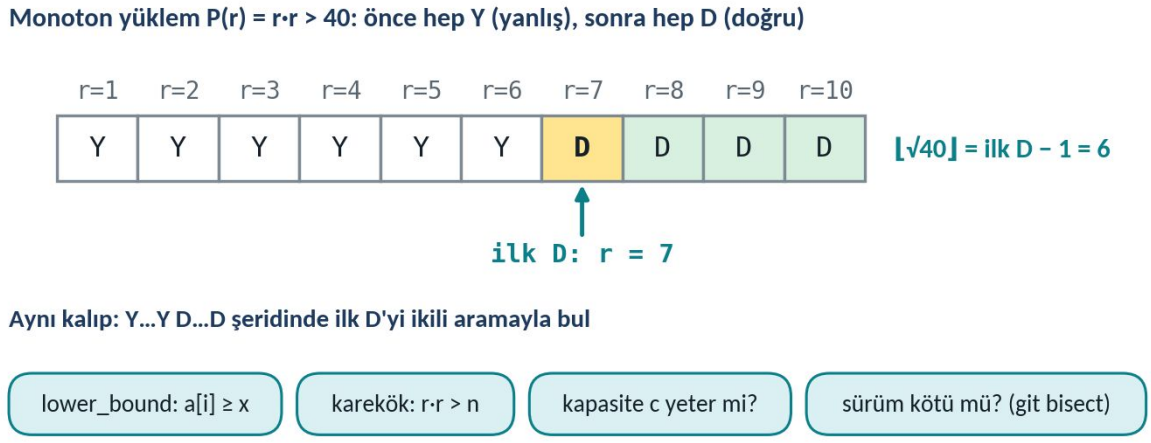
Şekil 4.3
Aynı dizide x = 61: kapalı aralık 4, yarı açık aralık 3 yoklamada bulur; daire içindeki sayılar yoklama sırasıdır (Örnek 3 ve 4)



3.4 Monoton yüklem: "ilk doğru"yu aramak

İkili aramayı dizilerden bağımsız düşünmenin anahtarı **monoton yüklemdir** (monotone predicate): bir kez doğru olunca hep doğru kalan bir P koşulu. Değerleri Y … Y D … D dizilir ve aradığımız **ilk D**'dir. lower_bound bunun bir örneğidir: P(i) = "a[i] ≥ x". P'nin dizi elemanına bakması gerekmez: P(r) = "r·r > 40" şeridi Y Y Y Y Y D D D D biçimindedir, ilk D r = 7'dir ve ⌊√40⌋ = 6 (Şekil 4.4). "Bu kapasite yeter mi?", "bu sürümde hata var mı?", "f(m) ile f(a) aynı işaretli mi?" soruları da monotondur. Yanıtı hesaplamak zor ama bir adayı sınamak kolaysa yanıt aralığında ikili arama yapılır (Bölüm 4.4).

Şekil 4.4
Monoton yüklem şeridi P(r) = r·r > 40 (Örnek 10): ilk D r = 7, ⌊√40⌋ = 6. Altta aynı kalıbın dört kullanımı



3.5 Kaputun Altında

> KAPUTUN ALTINDA · **Küçük dizide neden doğrusal arama kazanır? (bu makinede ölçüldü)**
>
> Karmaşıklık işlem **sayısını**, donanım işlem **başına süreyi** belirler. n = 10'da C -O2 doğrusal arama ortanca 10,2 ns, ikili arama 16,5 ns sürdü (oran 0,62); n = 20'de doğrusal arama 15,2, ikili arama 4,1 karşılaştırma yaptığı hâlde doğrusal arama yine hızlıydı. Kalıcı kesişim n ≈ 43 (40 ile 50 arası; Bölüm 6).
>
> Olası açıklama (performans sayaçlarıyla ölçülmedi): doğrusal döngü ardışık adreslere erişir, dalı hep aynı yöne gider ve işlemcinin **dal tahmini** (branch prediction) neredeyse hiç şaşmaz. İkili aramada "sola mı sağa mı?" sonucu veriye bağlıdır ve rastgele sorgularda yarı yarıya değişir; her şaşan tahmin onlarca saat döngüsüne mal olur.
>
> **Önbellek** büyük n'de görünür: C -O2'de yoklama başına süre n = 10^3'te 5,6 ns, 10^5'te 6,1 ns, 10^6'da 10,1 ns, 10^7'de 24,8 ns. L2 önbelleği çekirdek başına 1 MiB, L3 33 MiB'tır (`deney/OLCUM_ORTAMI.txt`); int dizisi n ≈ 2,6·10^5'te L2'yi, n ≈ 8,7·10^6'da L3'ü aşar. Sığmayınca ikili aramanın uzak yoklamaları belleğe gider; doğrusal aramanın ardışık erişimini ise donanım önceden getirir.

> KAPUTUN ALTINDA · **Orta nokta ve öge adresi: iki aritmetik ayrıntı**
>
> **Orta nokta.** C ve C++'ta `int` toplamının taşması **tanımsız davranıştır** (undefined behavior; ISO/IEC, 2018): derleyici taşmanın olmayacağını varsayabilir. `lo + (hi - lo) / 2` taşmaz; Python'da `(lo + hi) // 2` her zaman doğrudur (Bölüm 5.1, Hata Avcısı 3).
>
> **Öge adresi.** `bsearch` ögenin türünü bilmez; `void *` taban, öge sayısı ve boyutuyla mid. ögenin adresini AP-1 H10'daki formülle bayt cinsinden hesaplar: taban + mid × boyut. Kavram 10'a göre Örnek 12'deki `struct ogrenci` bu makinede 32 bayttır; ögeler taban + 0, 32, 64, … baytta durur (alanlar 28 bayt, dolgu 4 bayt; Bölüm 5.4).

4. Algoritmalar

Amiral algoritmaları aynı on bir adımla işliyoruz: problem, fikir, elle izleme, sözde kod, doğruluk, analiz (kimlik kartıyla), C gerçekleştirimi, üç dil karşılaştırması, test, deney, varyantlar. Üç dil kesitlerinde C++ ve Python'un C'den farklılaşan satırları gösterilir; hardal satırlar anlam farkıdır. Python fonksiyonlarının tamamı ve dile özgü notlar Ek A'dadır; C++ çekirdek fonksiyonları C'ninkilerle satır satır aynı yapıdadır.

4.1 Doğrusal arama ve nöbetçi

1 · Problem. Sırası bilinmeyen a[0..n − 1] içinde x'in **ilk** konumu, yoksa −1 (Python'da `None`). **2 · Fikir.** Dağınık raftaki bir kitabı ararken her kitaba sırayla bakarız. Sırasız dizide daha iyisi yoktur: x'in yok olduğunu söylemeden önce n elemanın hepsine bakmak zorunludur. **3 · Elle izleme.** Örnek 1'de a = [7, 3, 9, 3, 5, 1, 8]: `x = 3: konum 1, karşılaştırma 2` (3 iki kez geçer, ilk konum verilir); `x = 6: bulunamadı, karşılaştırma 7`. Klasik döngü her adımda **iki** karşılaştırma yapar: `i < n` ve `a[i] == x`. **Nöbetçi** (sentinel) tekniği x'i dizinin sonundaki boş yere (a[n]) yazar; döngü x'i mutlaka bulacağından sınır denetimi döngüden çıkar ve sonda bir kez "konum n mi?" diye bakılır.

SÖZDE KOD · 4 · Sözde kod: doğrusal arama ve nöbetçili sürümü

```
algoritma DOĞRUSAL-ARA(a[0..n−1], x)
    i ← 0'dan n−1'e kadar her i için    ▷ değişmez: x, a[0..i−1] içinde yok
        eğer a[i] = x ise döndür i
    döndür −1
algoritma NÖBETÇİLİ-ARA(a[0..n], x)      ▷ a[n] yazılabilir boş yer
    eski ← a[n];  a[n] ← x;  i ← 0
    a[i] ≠ x iken i ← i + 1              ▷ sınır denetimi yok
    a[n] ← eski;  eğer i < n ise döndür i; değilse döndür −1
```

Şekil 4.5
Başarısız aramada (x = 99, n = 10) karşılaştırma sayısı: klasik döngüde 2n + 1 = 21, nöbetçili döngüde n + 2 = 12; nöbetçi a[n]'ye yazılır (Örnek 2)



5 · Doğruluk. Değişmez (kart): x, a[0..i − 1] içinde yok. Başlatmada a[0..−1] boştur; gövde a[i] = x ise döner, dönmezse değişmez a[0..i] için de doğrudur; döngü ya ilk eşitlikte ya da i = n'de biter ve i = n'de değişmez "x yok" der. Nöbetçili döngü a[n] = x olduğu için **mutlaka** sona erer. **6 · Analiz.** En iyi 1, en kötü n karşılaştırma; konumlar eşit olasılıklıysa başarılı arama ortalama (n + 1)/2 (Örnek 1, n = 1000: 500,50;

başarısız 1000,00). Sıralı dizide a[i] ≥ x görülünce durmak başarısız aramayı da kısaltır: 501,00. Θ(n) zaman, O(1) ek bellek (Tablo 4.2).

Tablo 4.2
Doğrusal arama kimlik kartı

ALGORİTMA KİMLİK KARTI **Doğrusal arama** *(linear search; sentinel variant)*	
Strateji	Kaba kuvvet (tek geçiş; erken çıkış; nöbetçi tekniği)
Girdi önkoşulu	Yok — dizi sıralı olmak zorunda değil (sıralıysa a[i] > x görülünce erken çıkılabilir)
Zaman (en iyi · ortalama · en kötü)	O(1) · Θ(n) · Θ(n)
Ek bellek	O(1)
Özellikler	Başarılı aramada ortalama (n + 1)/2, başarısızda n karşılaştırma (n = 1000: 500,50 / 1000,00). Nöbetçi (x, a[n]'ye yazılır) sınır denetimini döngüden çıkarır: başarısız aramada toplam karşılaştırma 2n + 1'den n + 2'ye iner (n = 10^6: 2 000 001 → 1 000 002); dizinin sonunda yazılabilir bir boş yer gerekir
Değişmez / doğruluk	Döngü başında x, a[0..i−1] içinde yok
Kütüphane karşılığı	C: yok (elle; POSIX `lfind`) · C++: `std::find` (bulamazsa `end()`) · Python: `x in a`, `a.index(x)` (bulamazsa `ValueError`)
Ne zaman kullanılır	Veri sırasızsa; tek seferlik aramada; küçük dizilerde (bu makinede ikili arama C -O2'de n ≈ 43, C++ -O2'de n ≈ 59 sonrasında öne geçti; Python'da küçük n'de başa baş); yalnızca sıralı erişilen yapılarda (bağlı liste, dosya akışı)
Ne zaman kullanılmaz	Büyük, sıralı veride çok sayıda sorgu yapılacaksa (ikili arama; anahtar–değer için karma tablosu, H10)

7 · C gerçekleştirimi. (Örnek 1 ve 2). Nöbetçi diziyi geçici olarak **değiştirdiği** için parametre `const` olamaz ve kapasite n + 1 olmalıdır; 46. satır diziyi geri yükler (`a[10] (nöbetçi yeri) aramalardan sonra: 0`).

C `c/H04_01_dogrusal_arama.c · satır 19–29`

```c
static long dogrusal_ara(const int a[], size_t n,
                         int x)
{
    for (size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {
            return (long)i;
        }
    }
    return -1;
}
```

C `c/H04_02_nobetcili_arama.c · satır 36–49`

```c
static long nobetcili_ara(int a[], size_t n, int x)
{
    int eski = a[n];
    a[n] = x;               /* nöbetçi */
    size_t i = 0;
```

```
    eleman_kars++;
    while (a[i] != x) {
        i++;
        eleman_kars++;
    }
    a[n] = eski;            /* geri yükle */
    indis_kars++;           /* tek sınır denetimi */
    return i < n ? (long)i : -1;
}
```

8 · Üç dil karşılaştırması. C++'ta dizi `const std::vector<int>&` ile kopyalanmadan aktarılır, boyut `a.size()`'dır; Python'da `len(a)`, `None` ve modül düzeyi sayaç için `global` kullanılır:

C++ H04_01_dogrusal_arama.cpp · satır 22–25

```
static long dogrusal_ara(const std::vector<int>& a,
                         int x)
{
    for (std::size_t i = 0; i < a.size(); i++) {
```

Python H04_01_dogrusal_arama.py · satır 16–22

```
def dogrusal_ara(a, x):
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1
        if a[i] == x:
            return i
    return None
```

9 · Test. Boş dizi, ilk/son eleman, tekrar eden ve olmayan değer; kâhin (test oracle) `std::find`, `x in a`. Örnek 2, 2n + 1 ile n + 2'yi n = 10 … 10^6'da birebir doğrular. **10 · Deney.** Nöbetçili/klasik süre oranı n = 10^5'te C -O2'de 0,49, Python'da 1,03 (Bölüm 6). **11 · Varyantlar.** Son konum, bütün konumlar, sıralı dizide erken çıkış, bağlı liste (H8). Dizi sıralıysa bir karşılaştırma yarım diziyi eleyebilir.

4.2 İkili arama

1 · Problem. Artan sıralı a[0..n − 1] içinde x'in bir konumu, yoksa −1. Örnek 3–5'in ortak dizisi a = [3, 8, 14, 19, 23, 31, 37, 42, 50, 56, 61, 67, 73, 80, 88, 95] (n = 16). **2 · Fikir.** Rehberde "Kaya"yı ararken ortadan açarız; açılan sayfada "M" görürsek arkadaki yarıya bir daha bakmayız. İkili arama (binary search) aday bölgenin ortasındaki a[mid]'i inceler ve yanıta göre bölgenin yarısını atar. **3 · Elle izleme.** Şekil 4.6 x = 61 için dört adımı gösterir (soluk gri: elenen, sarı: yoklanan a[mid]). Bulunamayan x = 45'in adımlarını Tablo 4.3 verir: son adımda lo = 8 > hi = 7 olur ve aday bölge boşalır.

SÖZDE KOD · 4 · Sözde kod: kapalı aralıklı ikili arama

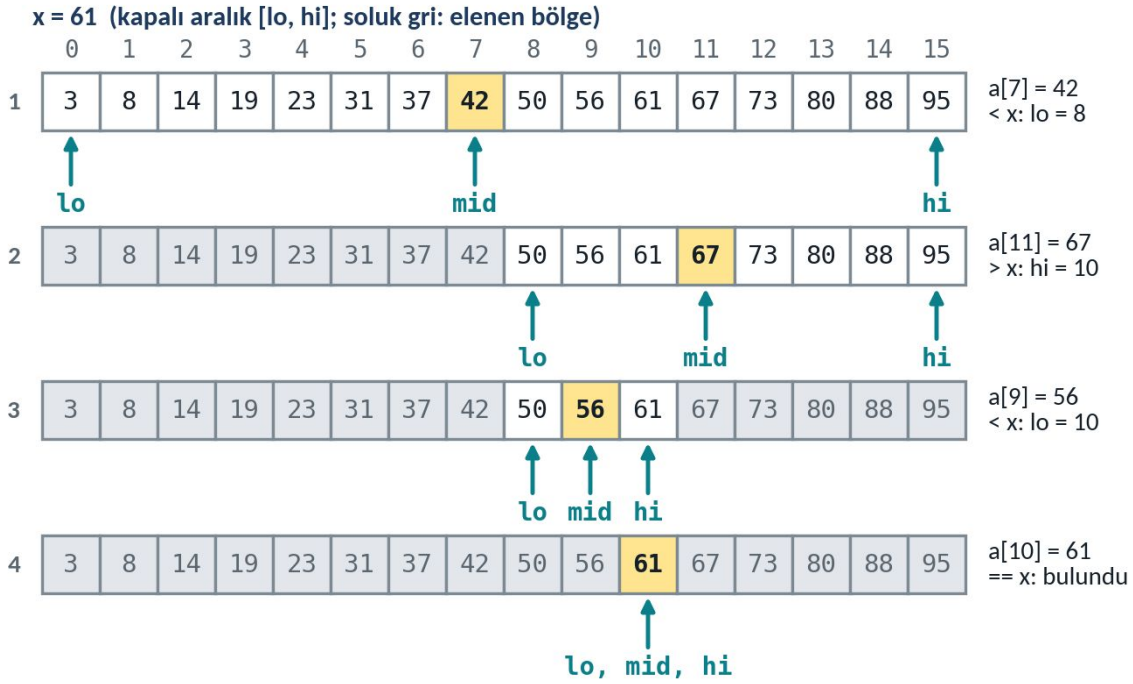
```
algoritma İKİLİ-ARA(a[0..n−1], x)        ▷ ön koşul: a artan sıralı
    lo ← 0;  hi ← n − 1                  ▷ aday bölge a[lo..hi]
    lo ≤ hi iken
        mid ← lo + ⌊(hi − lo) / 2⌋       ▷ taşmasız orta nokta
        eğer a[mid] = x ise döndür mid
        eğer a[mid] < x ise lo ← mid + 1 ▷ a[lo..mid] elendi
        değilse hi ← mid − 1             ▷ a[mid..hi] elendi
    döndür −1                            ▷ lo = hi + 1: aday kalmadı
```

Tablo 4.3
x = 45 için kapalı aralıklı ikili aramanın izlemesi (Örnek 3; son durumda lo > hi)

adım	lo	hi	mid	a[mid]	karar	elenen indisler	kalan [lo, hi]
1	0	15	7	42	< x: lo = 8	0..7	[8, 15]
2	8	15	11	67	> x: hi = 10	11..15	[8, 10]
3	8	10	9	56	> x: hi = 8	9..10	[8, 8]
4	8	8	8	50	> x: hi = 7	8..8	[8, 7]

> TAHMİN ET · **Sizce sırada ne olacak?**
> x = 45 aranırken üçüncü adımdan sonra lo = hi = 8'dir. Hangi eleman yoklanır, arama nasıl biter? (Yanıt: mid = 8, a[8] = 50 > 45, hi = 7; lo > hi, −1 döner. 45, a[7] = 42 ile a[8] = 50 arasına girerdi.)

Şekil 4.6
İkili arama aralık daralma şeridi, x = 61 (kapalı aralık): lo/mid/hi okları turkuaz, soluk gri elenen bölge, sarı yoklanan eleman (Örnek 3'ün adımları)



5 · Doğruluk. Değişmez: "x dizideyse a[lo..hi] içindedir". Başlatmada bölge tüm dizidir. a[mid] < x ise, dizi sıralı olduğundan a[lo..mid]'in hiçbiri x olamaz ve lo = mid + 1 değişmezi korur; a[mid] > x için simetrik. lo ≤ mid ≤ hi olduğundan iki güncelleme de mid'i bölgeden çıkarır: aday sayısı hi − lo + 1 **kesin** azalır ve döngü ya eşitlikte ya da lo = hi + 1'de (aday yok) biter. **6 · Analiz.** Bir adımda m aday en çok ⌊m/2⌋'ye iner: T(n) = T(⌊n/2⌋) + c, T(0) = 0; yerine koymayla k = ⌊log₂ n⌋ + 1 adımda ⌊n/2ᵏ⌋ = 0 olur. En çok ⌊log₂ n⌋ + 1 yoklama, Θ(log n), O(1) ek bellek (Cormen vd., 2022; Knuth, 1998). Örnek 3 sınırı n = 1 … 4096'da **bütün** başarılı ve başarısız sorgularla denetler: n = 1023'te en çok 10, n = 1024'te 11 yoklama (sınır n ikinin kuvvetini geçerken bir artar); n = 4096'da başarılı aramanın ortalaması 11,003, sınırın yaklaşık bir altında, çünkü elemanların yaklaşık yarısı karar ağacının en alt düzeyindedir.

Tablo 4.4
İkili arama kimlik kartı

ALGORİTMA KİMLİK KARTI **İkili arama** *(binary search)*	
Strateji	Azalt ve fethet (her adımda aday aralık yarıya iner)
Girdi önkoşulu	Dizi artan sıralı ve rastgele erişimli (a[i]'ye O(1) erişim)
Zaman (en iyi · ortalama · en kötü)	O(1) · Θ(log n) · Θ(log n)
Ek bellek	O(1) yinelemeli / O(log n) özyinelemeli
Özellikler	En çok $\lfloor \log_2 n \rfloor + 1$ yoklama (n = 10^6: 20); T(n) = T(n/2) + c. Aralık sözleşmesi tek yerde seçilir: kapalı [lo, hi] (hi = n − 1, lo <= hi, hi = mid − 1) ya da yarı açık [lo, hi) (hi = n, lo < hi, hi = mid). Orta nokta taşmasız: lo + (hi − lo) / 2
Değişmez / doğruluk	x dizideyse a[lo..hi] içindedir; a[0..lo−1] < x ve a[hi+1..n−1] > x
Kütüphane karşılığı	C: `bsearch` (`void *` taban + karşılaştırıcı) · C++: `std::binary_search` (var/yok), `std::lower_bound` · Python: `bisect.bisect_left` + eşitlik denetimi
Ne zaman kullanılır	Sıralı, rastgele erişimli veride çok sayıda arama; n büyükken (bu makinede C -O2'de n ≈ 43 sonrasında doğrusal aramadan hızlı; Python'da küçük n'de başa baş)
Ne zaman kullanılmaz	Veri sıralı değilse (önce sıralama: H5–H7) ya da sık ekleme/silme varsa (her ekleme O(n) kaydırma; dengeli arama ağaçları Veri Yapıları dersinde); çok küçük dizilerde

7 · C gerçekleştirimi. (Örnek 3). `lo`, `hi` ve `mid` işaretli `long`'dur: boş dizide hi = −1 olur; `size_t` −1'i en büyük işaretsiz sayıya sarardı (Hata Avcısı 9).

C `c/H04_03_ikili_arama.c · satır 20–35`

```c
static long ikili_ara(const int a[], size_t n, int x)
{
    long lo = 0, hi = (long)n - 1;  /* hi = -1 olabilir: işaretli */
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2; /* taşmasız */
        yoklama++;
        if (a[mid] == x) {
            return mid;
        } else if (a[mid] < x) {
            lo = mid + 1;   /* a[lo..mid] elendi */
        } else {
            hi = mid - 1;   /* a[mid..hi] elendi */
        }
    }
    return -1;                          /* aralık boş: lo == hi + 1 */
}
```

8 · Üç dil karşılaştırması. Döngü üç dilde satır satır aynıdır. Python'da `(lo + hi) // 2` güvenlidir (tamsayı taşmaz) ve 0 ≤ lo ≤ hi iken lo + ⌊(hi − lo)/2⌋ ile aynı değeri verir:

C++ `H04_03_ikili_arama.cpp · satır 23–25`

```cpp
    long lo = 0, hi = static_cast<long>(a.size()) - 1;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2; // taşmasız
```

Python **H04_03_ikili_arama.py · satır 18–20**

```python
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2    # taşma yok
```

9 · Test. Boş dizi (`boş dizi: ikili_ara(7) = -1 yoklama = 0`), tek eleman, ilk/son eleman, dış değerler; Örnek 3 sonucu formülle (a[i] = 2i + 1), Örnek 12 kütüphane kâhinleriyle karşılaştırır. **Yarı açık sözleşme (Örnek 4)** değişmezi ve sonlanmayı her adımda `assert` ile denetler (denetim O(n): yalnızca geliştirmede):

C **c/H04_04_yari_acik_aralik.c · satır 45–58**

```c
    size_t lo = 0, hi = n;              /* başlangıç: tüm dizi [0, n) */
    while (lo < hi) {                   /* aralık boş değil */
        assert(degismez_mi(a, n, lo, hi, x));
        size_t mid = lo + (hi - lo) / 2;    /* lo <= mid < hi */
        size_t eski = hi - lo;
        yoklama++;
        if (a[mid] == x) {
            return (long)mid;
        } else if (a[mid] < x) {
            lo = mid + 1;               /* a[lo..mid] < x */
        } else {
            hi = mid;                   /* a[mid..hi-1] > x */
        }
        assert(hi - lo < eski);         /* sonlanma: aralık daraldı */
```

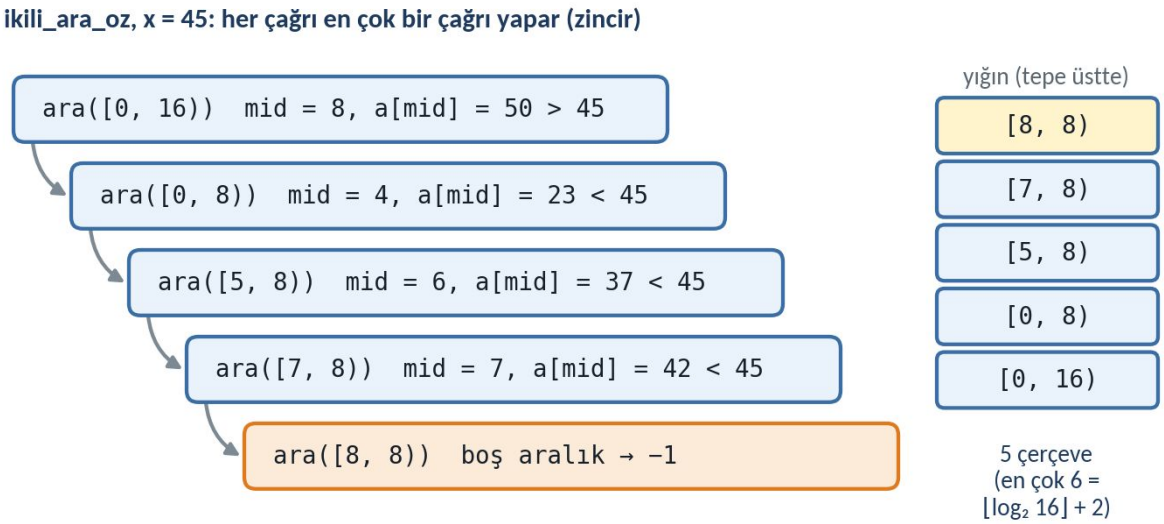
n = 0 … 64 ve x = −1 … 2n + 1 için 4 355 sorguda hatalı sonuç 0; değişmez 23 807 kez denetlendi ve hiç bozulmadı. `bozuk` argümanıyla sırasız dizide değişmez ilk adımda bozulur ve süreç durur: `c/H04_04_yari_acik_aralik.c:47: ikili_ara_yari: Assertion degismez_mi(a, n, lo, hi, x) failed.` (çıkış kodu 134); `-DNDEBUG` ile derlenince `assert`'ler kaybolur. **10 · Deney.** 10^6 elemanda 1000 rastgele sorguda en çok 20 yoklama görüldü ($\lfloor \log_2 n \rfloor + 1 = 20$); süreler Bölüm 6'dadır.

11 · Varyantlar. Özyinelemeli sürüm (Örnek 5) aday bölgeyi parametre olarak taşır; T(n) = T(n/2) + c doğrudan koddan okunur. Her çağrı ya döner ya da **tek** özyinelemeli çağrı yapar: ağaç bir zincirdir (Şekil 4.7) ve en kötü yol $\lfloor \log_2 n \rfloor + 2$ çağrıdır (son çağrı boş aralık; n = 4096: 14 çağrı, derinlik 14). Yığın belleği O(log n), yinelemeli sürümde O(1).

C **c/H04_05_ozyinelemeli_ikili.c · satır 25–35**

```c
    if (lo >= hi) {                     /* temel durum: boş aralık */
        return -1;
    }
    size_t mid = lo + (hi - lo) / 2;
    if (a[mid] == x) {                  /* temel durum: bulundu */
        return (long)mid;
    }
    if (a[mid] < x) {                   /* sağ yarı: [mid + 1, hi) */
        return ikili_ara_oz(a, mid + 1, hi, x, derinlik + 1);
    }
    return ikili_ara_oz(a, lo, mid, x, derinlik + 1);   /* [lo, mid) */
```

Şekil 4.7
Özyinelemeli ikili arama, x = 45 (Örnek 5): her çağrı en çok bir çağrı yapar; en derin anda yığında beş çerçeve vardır (n = 16'da en kötü durum ⌊log₂ 16⌋ + 2 = 6)

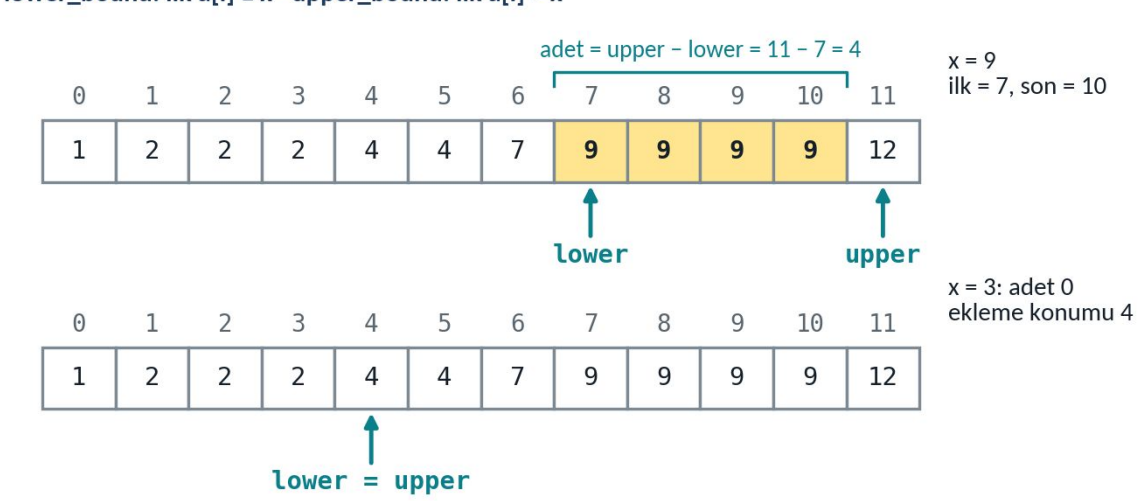


Kaputun altında: iki özyinelemeli çağrı da **kuyruk konumundadır** (tail position). GCC 13.3 -O2 ile derlenmiş `ikili_ara_oz` gövdesinde `call ikili_ara_oz` 0 kez, -O0 ile 2 kez geçer: -O2'de çağrı bir sıçramaya (`jmp`) dönüşmüştür. C standardı bunu güvence etmez, Python hiç yapmaz; derinlik ⌊log₂ n⌋ + 2 olduğundan (10^6 için 21) Python'un 1000'lik sınırı sorun değildir.

4.3 lower_bound ve upper_bound: tekrar eden anahtarlar

1 · Problem. a = [1, 2, 2, 2, 4, 4, 7, 9, 9, 9, 9, 12] gibi tekrar eden değerli sıralı dizide x = 9'un **ilk** ve **son** konumu, kaç kez geçtiği, x yoksa nereye ekleneceği. Kapalı aralıklı arama eşitlerden herhangi birinde durur. **2 · Fikir. lower_bound** ilk a[i] ≥ x, **upper_bound** ilk a[i] > x konumudur (hiç yoksa n). Arama eşitlikte **durmaz**: lower_bound a[mid] ≥ x görünce "ilk konum mid ya da solunda" der. Adet = upper − lower, son konum = upper − 1; x yoksa lower = upper ekleme konumudur (Şekil 4.8). İki fonksiyon tek bir işaretle ayrılır: `<` ile `<=`.

Şekil 4.8
Tekrar eden değerlerde lower_bound ve upper_bound (Örnek 7): x = 9 için 7 ve 11 (adet 4); dizide olmayan x = 3 için ikisi de 4 (ekleme konumu)



3 · Elle izleme. Tablo 4.5 x = 9 için iki aramayı yan yana gösterir. İlk adım aynıdır (a[6] = 7 < 9 ve 7 ≤ 9); ikinci adımda a[9] = 9 için lower_bound "≥ x" der ve sola (hi = 9), upper_bound "≤ x" der ve sağa (lo = 10) gider.

Tablo 4.5
lower_bound(9) = 7 ve upper_bound(9) = 11 izlemesi (Örnek 7)

adım	lo	hi	mid	a[mid]	lower: karar	lo	hi	mid	a[mid]	upper: karar
1	0	12	6	7	< x: lo = 7	0	12	6	7	<= x: lo = 7
2	7	12	9	9	**>= x: hi = 9**	7	12	9	9	**<= x: lo = 10**
3	7	9	8	9	>= x: hi = 8	10	12	11	12	> x: hi = 11
4	7	8	7	9	>= x: hi = 7	10	11	10	9	<= x: lo = 11

> **TAHMİN ET · Sizce sonuç ne?**
>
> Aynı dizide x = 4 için lower_bound ve upper_bound kaçtır? (Yanıt: 4 ve 6; adet 2. x = 5 için ikisi de 6: 5 yok, a[6] = 7'nin önüne girer.)

SÖZDE KOD · 4 · Sözde kod: lower_bound (ve upper_bound farkı)

```
algoritma LOWER-BOUND(a[0..n−1], x)      ▷ ilk a[i] ≥ x; hepsi < x ise n
    lo ← 0;  hi ← n
    lo < hi iken                          ▷ değişmez: a[0..lo−1] < x ≤ a[hi..n−1]
        mid ← lo + ⌊(hi − lo) / 2⌋
        eğer a[mid] < x ise lo ← mid + 1
        değilse hi ← mid                  ▷ eşitlikte durma: sola devam
    döndür lo
▷ UPPER-BOUND: 5. satırda a[mid] ≤ x; değişmez a[0..lo−1] ≤ x < a[hi..n−1]
```

5 · Doğruluk. Değişmez: a[0..lo − 1] < x ≤ a[hi..n − 1]; başlatmada iki dış bölge boştur. a[mid] < x ise a[lo..mid]'in hepsi < x'tir (lo = mid + 1); a[mid] ≥ x ise a[mid..hi − 1]'in hepsi ≥ x'tir (hi = mid). Döngü lo == hi ile biter ve a[lo], ≥ x olan ilk elemandır. **6 · Analiz.** Erken çıkış olmadığından her arama $\lfloor \log_2 n \rfloor$ ya da $\lfloor \log_2 n \rfloor + 1$ yoklama sürer (n = 12'de bütün sorgular 3 ya da 4). Bir değerin adedi iki aramadır, Θ(log n): b[i] = ⌊i/10⌋ olan 1 000 000 elemanlı dizide 12345 değeri 10 kez geçer (konum 123450..123459) ve iki arama bunu toplam 39 yoklamada bulur; doğrusal sayım Θ(n)'dir.

Tablo 4.6
lower_bound / upper_bound kimlik kartı

ALGORİTMA KİMLİK KARTI	**lower_bound / upper_bound** *(binary search variants)*
Strateji	Azalt ve fethet (eşitlikte durmayan ikili arama)
Girdi önkoşulu	Dizi artan sıralı; tekrar eden anahtarlar olabilir
Zaman (en iyi · ortalama · en kötü)	Θ(log n) · Θ(log n) · Θ(log n)
Ek bellek	O(1)

Özellikler	Erken çıkış yoktur: her arama $\lfloor \log_2 n \rfloor$ ya da $\lfloor \log_2 n \rfloor + 1$ yoklama sürer. lower_bound ilk a[i] >= x, upper_bound ilk a[i] > x konumudur; adet = upper − lower, ilk konum = lower, son konum = upper − 1; x yoksa lower ekleme konumudur (n = 10^6'da bir değerin 10 kopyası 39 yoklamada sayıldı)
Değişmez / doğruluk	lower_bound: a[0..lo−1] < x ≤ a[hi..n−1]; upper_bound: a[0..lo−1] ≤ x < a[hi..n−1]
Kütüphane karşılığı	C: yok (`bsearch` eşitlerden herhangi birini döndürür) · C++: `std::lower_bound`, `std::upper_bound`, `std::equal_range` · Python: `bisect.bisect_left`, `bisect.bisect_right`
Ne zaman kullanılır	Tekrar eden anahtarlarda konum ve sayım, aralık sorguları (lo ≤ x < hi kaç tane?), ekleme konumu, önek araması (Mini-OBS 2 v2.3)
Ne zaman kullanılmaz	Sırasız veride; yalnızca var/yok sorusunda (üç yönlü ikili arama ya da karma tablosu daha doğrudan)

7 · C gerçekleştirimi. (Örnek 7). `ust_sinir` aynı fonksiyondur; yalnızca 43. satırdaki karşılaştırma `a[mid] <= x` olur:

C `c/H04_07_alt_ust_sinir.c · satır 20–33`

```c
static size_t alt_sinir(const int a[], size_t n, int x)
{
    size_t lo = 0, hi = n;
    while (lo < hi) {
        size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}
```

8 · Üç dil karşılaştırması. Döngü üç dilde aynıdır; C++'ta konum `std::size_t`, Python'da sınırsız tamsayıdır. Asıl fark kütüphanededir: C'de lower_bound yoktur (`bsearch` eşitlerden herhangi birini döndürür), `std::lower_bound` yineleyici, `bisect_left` indis döndürür (Bölüm 8):

C++ `H04_07_alt_ust_sinir.cpp · satır 21–23`

```cpp
static std::size_t alt_sinir(const std::vector<int>& a, int x)
{
    std::size_t lo = 0, hi = a.size();
```

Python `H04_07_alt_ust_sinir.py · satır 20–22`

```python
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
```

9 • Test. Kâhin: doğrusal sayım, `std::lower_bound`/`upper_bound`, `bisect_left`/`bisect_right` (`Kâhinle karşılaştırma: 200 rastgele dizi, 10000 sorgu, uyuşmazlık 0`); sınırlar: x < a[0], x > a[n − 1], dizinin tamamı x. **10 • Deney.** Erken çıkışsız aramanın yoklaması $\lfloor \log_2 n \rfloor + 1$'e çok

yakındır (Bölüm 6). **11 · Varyantlar.** Ekleme konumu (5.2), önek araması (Örnek 12, Mini-OBS 2), `std::equal_range`, aralık sorgusu (Atölye 3). lower_bound, "a[i] ≥ x" yükleminin **ilk doğru**sudur; yüklemin dizi elemanı olması gerekmez.

4.4 Cevap üzerinde ikili arama

1 · Problem. $0 \le n \le 2^{64} - 1$ için $\lfloor\sqrt{n}\rfloor$'yi kayan nokta kullanmadan bulmak (`sqrt`'ün double'ı 53 bitlik anlamlı kısmıyla büyük n'de yetmez). **2 · Fikir.** P(r) = "r·r > n" monotondur (Bölüm 3.4) ve $\lfloor\sqrt{n}\rfloor$ = ilk D − 1'dir. Dizide değil, olası yanıtların [lo, hi] aralığında ararız: **cevap üzerinde ikili arama** (binary search on the answer). Sayı tahmin oyunu da budur: tutulan sayı yanıt, "küçük/büyük" yüklemin değeridir. **3 · Elle izleme.** P(r) = r·r > 40 şeridinde (Şekil 4.4) lo = 1, hi = 40 ile başlayan arama ilk D'yi (r = 7) bulur, yanıt 6. Tahmin oyununda tutulan 35 için:

ÇIKTI · ÖRNEK 10 (3. BÖLÜMÜN İLK OYUNU)

```
  oyun 1: tutulan 35
    soru 1: 50 mi? -> daha küçük
    soru 2: 25 mi? -> daha büyük
    soru 3: 37 mi? -> daha küçük
    soru 4: 31 mi? -> daha büyük
    soru 5: 34 mi? -> daha büyük
    soru 6: 35 mi? -> doğru
    6 soruda bulundu
```

SÖZDE KOD · 4 · Sözde kod: monoton yüklemde ilk doğru

```
1  algoritma İLK-DOĞRU(lo, hi, P)            ▷ P monoton (Y…Y D…D) ve P(hi) = D
2      lo < hi iken                           ▷ değişmez: P(lo − 1) = Y, P(hi) = D
3          mid ← lo + ⌊(hi − lo) / 2⌋
4          eğer P(mid) ise hi ← mid           ▷ ilk D mid ya da solunda
5          değilse lo ← mid + 1               ▷ ilk D mid'in sağında
6      döndür lo
7  ▷ ⌊√n⌋ = İLK-DOĞRU(1, min(n, 2³²), r ↦ r > ⌊n / r⌋) − 1   (n ≥ 2)
```

5 · Doğruluk. Değişmez: P(lo − 1) = Y, P(hi) = D. P(0) tanım gereği Y'dir; n ≥ 2 için P(n) = D ve 2^{32}'den büyük n'de P(2^{32}) = D'dir ($\lfloor n/2^{32} \rfloor < 2^{32}$). Güncellemeler lower_bound'daki gibidir. **Taşma:** `r * r > n` yerine `r > n / r` yazılır; tamsayılarda denktirler (r·r > n ⇔ r > ⌊n/r⌋) ama ilki 64 bitte r ≥ 2^{32}'de taşar. **6 · Analiz.** Yanıt aralığı R genişliğindeyse $\lceil \log_2 R \rceil$ yüklem değerlendirmesi; yüklem c maliyetliyse O(log R · c). n = $2^{64} - 1$ için $\lfloor\sqrt{n}\rfloor$ = 4 294 967 295, 32 adım. Tahmin oyununda en çok $\lfloor \log_2 N \rfloor + 1$ soru.

Tablo 4.7
Cevap üzerinde ikili arama kimlik kartı

ALGORİTMA KİMLİK KARTI **Cevap üzerinde ikili arama** *(binary search on the answer; bisection method)*	
Strateji	Azalt ve fethet (monoton yüklemde ilk "doğru"nun aranması)
Girdi önkoşulu	Yanıt aralığında monoton bir yüklem: P(r) Y…Y D…D biçiminde (bir kez doğru olunca hep doğru)
Zaman (en iyi · ortalama · en kötü)	— · — · O(log R · c)
Ek bellek	O(1)

Özellikler	R: yanıt aralığının genişliği, c: yüklemin maliyeti. Tamsayı karekökte $R \le 2^{32}$: en çok 32 adım; sayı tahmin oyununda en çok $\lfloor \log_2 N \rfloor + 1$ soru ($N = 10^6$: 20); gerçel sayılarda ikiye bölme: tol toleransı için $\lceil \log_2((b-a)/tol) \rceil$ yineleme ($tol = 10^{-6}$: 20)
Değişmez / doğruluk	P(lo − 1) = Y ve P(hi) = D (ilk D [lo, hi] içindedir); ikiye bölmede f(a) ile f(b) ters işaretli
Kütüphane karşılığı	C: yok · C++: `std::partition_point` (yüklemle ikili arama) · Python: `bisect` + `key=` (3.10+); karekök için `math.isqrt`
Ne zaman kullanılır	Doğrudan çözülmesi zor ama "bu yanıt yeterli mi?" sorusu kolay ve monoton olan problemlerde (karekök, kapasite, "en büyüğün en küçüğü", denklem kökü)
Ne zaman kullanılmaz	Yüklem monoton değilse (yanlış yanıt); kayan noktada durma koşulu tam eşitlikse (sonsuz döngü: Hata Avcısı 8)

> TAHMİN ET · **Sizce kaçıncı soruda?**
>
> Tutulan 27 olsaydı ilk üç soru (50, 25, 37) aynı kalır. Dördüncü soru hangisidir, oyun kaç soru sürer? (Yanıt: aralık [26, 36], 4. soru 31; sonra 28, 26, 27: toplam 7 soru. Örnek 10 bu oyunu da oynar.)

7 · C gerçekleştirimi. (Örnek 10). Yüklem `cok_buyuk_mu(r, n)` tek satırdır: `return r > n / r;`.

c/H04_10_karekok_tahmin.c · satır 29–46

```c
static uint64_t tamsayi_karekok(uint64_t n)
{
    if (n < 2) {
        return n;
    }
    uint64_t lo = 1;                          /* P(0) tanım gereği Y */
    uint64_t hi = n < 4294967296u ? n : 4294967296u;    /* P(hi) = D */
    while (lo < hi) {
        uint64_t mid = lo + (hi - lo) / 2;
        adim++;
        if (cok_buyuk_mu(mid, n)) {
            hi = mid;       /* ilk D: mid ya da solu */
        } else {
            lo = mid + 1;   /* ilk D: mid'in sağı */
        }
    }
    return lo - 1;
}
```

8 · Üç dil karşılaştırması. C++'ta `std::uint64_t`, Python'da sınırsız tamsayı; Python'da `r * r` taşmaz ama aynı yüklem biçimi korunur:

H04_10_karekok_tahmin.cpp · satır 34–37

```cpp
    std::uint64_t lo = 1;                         // P(0) tanım gereği Y
    std::uint64_t hi = n < 4294967296u ? n : 4294967296u;   // P(hi) = D
    while (lo < hi) {
        std::uint64_t mid = lo + (hi - lo) / 2;
```

Python **H04_10_karekok_tahmin.py · satır 18–19**

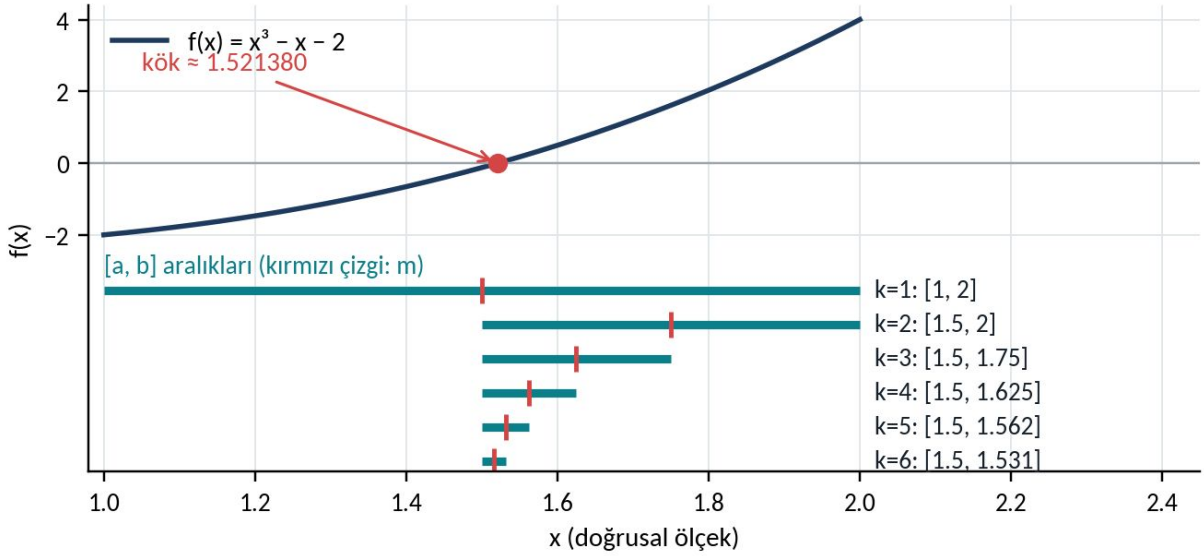
```
18  def cok_buyuk_mu(r, n):
19      return r > n // r
```

9 · Test. Her sonuç r·r ≤ n < (r + 1)·(r + 1) ile (taşmasız) doğrulanır, Python ayrıca `math.isqrt` kâhinini kullanır; sınırlar: 0, 1, tam kareler ve bir eksikleri, $2^{64} - 1$. **10 · Deney.** Örnek 10, 1..N'deki **her** sayıyı tutup oynar: N = 100'de soru dağılımı (1, 2, 4, 8, 16, 32, 37), ortalama 5,80; N = 10^6 için 1. … 19. soruda biten oyun sayıları tam 2^{k-1}'dir (dolu düzeyler), 20. soruda 475 713 oyun biter. k. soruda bulunan sayılar karar ağacının k. düzeyidir (2^{k-1} düğüm). **11 · Varyantlar. Kapasite problemleri** ("en büyüğün en küçüğü"). Kavram 7'de 12 paket sırayla en çok 4 seferde taşınacak; en küçük kapasite nedir? "c yeter mi?" yüklemi O(n)'de yanıtlanır ve monotondur (c yetiyorsa c + 1 de yeter); yanıt aralığı [en ağır paket, toplam] içinde ilk D aranır (`kavram/H04_K07_kapasite.c`). İkili arama 7 yüklem çağrısıyla kapasite 38'i (4 sefer) bulur; c'yi birer artıran kâhin aynı yanıta 20 çağrıda ulaşır: O(n log R) ile O(n·R). Gerçel sayılarda aynı fikir ikiye bölme (4.5), sürüm geçmişinde git bisect olur (Bölüm 9).

4.5 Gerçel sayılarda ikili arama: ikiye bölme yöntemi

1 · Problem. f(x) = $x^3 - x - 2$ = 0'ın [1, 2]'deki kökünü 10^{-6} duyarlıkla bulmak. **2 · Fikir.** f sürekli, f(1) = −2 < 0, f(2) = 4 > 0: ara değer teoremine göre aralıkta kök vardır. m'de f(m)'nin işareti kökün hangi yarıda olduğunu söyler: **ikiye bölme yöntemi** (bisection method). Yüklem "f(m) ile f(a) aynı işaretli mi?"dir. **3 · Elle izleme.** k = 1: m = 1,5, f(m) = −0,125 < 0 → a ← m; k = 2: m = 1,75, f(m) = 1,609 > 0 → b ← m; k = 3: m = 1,625, f(m) = 0,666 > 0 → b ← m; … Şekil 4.9 ilk altı yinelemenin aralıklarını gösterir.

Şekil 4.9
İkiye bölme yöntemi, f(x) = $x^3 - x - 2$, [1, 2] (Örnek 11): eğri, kök ve ilk altı yinelemenin daralan [a, b] aralıkları (çıktı satırlarından)



5 · Doğruluk. Değişmez: f(a) ile f(b) ters işaretlidir, [a, b] bir kök içerir; m'de işaret f(a) ile aynıysa kök [m, b]'de (a ← m), değilse [a, m]'dedir (b ← m). **6 · Analiz.** k yinelemede aralık (b − a)/2^k olur; tol'a inmek için $\lceil \log_2((b - a)/\text{tol}) \rceil$ yineleme gerekir (tol = 10^{-6}: 20). Örnek 11 tol = 10^{-2} … 10^{-20} için sayıyı formülle karşılaştırır: 10^{-15}'e kadar eşittir (50 yineleme); 10^{-16}'da formül 54 derken program 52 yinelemede "f(m) == 0" ile durur: bu yinelemede a ile b arasında tek bir double kalmıştır (genişlik $2 \cdot 2^{-52} \approx 4{,}4 \cdot 10^{-16}$) ve o noktada hesaplanan f(m) yuvarlamayla tam 0 çıkar. Öyle olmasaydı bir sonraki adımda yeni orta nokta kalmayacağı için "kayan nokta sınırı" ile durulurdu: [1, 2]'de komşu double'lar $2^{-52} \approx 2{,}2 \cdot 10^{-16}$ aralıklıdır, 10^{-16} toleransına hiç inilemez. Her ondalık basamak ≈ $\log_2 10 \approx 3{,}3$ yinelemedir.

7 · C gerçekleştirimi. (Örnek 11). Üç durma koşulu vardır: tolerans, f(m) == 0 ve m'nin a ya da b'ye eşit çıkması (komşu double'lar arasında yeni orta nokta yoktur):

C c/H04_11_ikiye_bolme.c · satır 35–58

```c
    while (b - a > tol && yineleme < EN_COK_YINELEME) {
        double m = a + (b - a) / 2;
        if (m <= a || m >= b) {         /* a ile b komşu double: dur */
            durma = "kayan nokta sınırı";
            break;
        }
        double fm = f(m);
        yineleme++;
        if (yaz) {
            printf("  %2d  %.9f  %.9f  %.9f  %10.3e\n", yineleme, a, b, m,
                   fm);
        }
        if (fm == 0.0) {                /* double'da tam sıfır */
            durma = "f(m) == 0";
            return m;
        }
        if ((fa < 0.0) == (fm < 0.0)) { /* kök sağ yarıda */
            a = m;
            fa = fm;
        } else {                        /* kök sol yarıda */
            b = m;
        }
    }
    return a + (b - a) / 2;
```

8 · Üç dil karşılaştırması. Python `float`'u C `double`'ı ile aynı IEEE 754 biçimidir ve işlemler aynı sırada yapıldığından üç dil **bit bit aynı** çıktıyı üretir. C++ sürümü C ile aynı yapıdadır (`bool yaz`, `std::printf`; durma nedeni `std::string`). Python'da orta nokta adımı (tam fonksiyonlar Ek A'da):

Python H04_11_ikiye_bolme.py · satır 31–33

```python
    while b - a > tol and yineleme < EN_COK_YINELEME:
        m = a + (b - a) / 2             # Python float = IEEE 754 double
        if m <= a or m >= b:            # a ile b komşu double: dur
```

9 · Test. `x^3 - x - 2, tol = 1e-15: f(kök) = 8.882e-16`; $x^2 - 2$ için tol = 0 ile 52 yinelemede "kayan nokta sınırı"nda durulur (tam sıfırı beklemek: Hata Avcısı 8). **10 · Deney.** Yineleme–tol ilişkisi yukarıdaki analizdedir. **11 · Varyantlar.** Fonksiyon işaretçisiyle genel ikiye bölme (Bölüm 13); Newton yöntemi.

4.6 Merdiven araması: satır ve sütunları sıralı matris

1 · Problem. Her satırı soldan sağa, her sütunu yukarıdan aşağıya artan m × n matriste x'i bulmak (Örnek 14, 5 × 6). **2 · Fikir.** Sağ üst köşedeki eleman satırının en büyüğü, sütununun en küçüğüdür. a[r][c] > x ise x bu sütunda olamaz: sütun elenir, sola gidilir. a[r][c] < x ise x bu satırda olamaz: satır elenir, aşağı inilir. Her adım bir satır **ya da** bir sütun eler; yol bir merdiven çizer. **3 · Elle izleme.** Şekil 4.10: x = 15 dört adımda bulunur; x = 19 8 adımda (17, 20, 17, 20, 17, 18, 21, 18) matristen çıkar.

SÖZDE KOD · 4 · Sözde kod: merdiven araması

```
algoritma MERDİVEN-ARA(a[0..m−1][0..n−1], x)
    r ← 0;  c ← n − 1                    ▷ sağ üst köşe
    r < m ve c ≥ 0 iken                   ▷ değişmez: x varsa a[r..m−1][0..c] içinde
        eğer a[r][c] = x ise döndür (r, c)
        eğer a[r][c] > x ise c ← c − 1     ▷ sütun c elendi
        değilse r ← r + 1                 ▷ satır r elendi
    döndür yok
```

5 · Doğruluk. Değişmez: x varsa a[r..m − 1][0..c] içindedir. Sola gidişte c. sütunun a[r..m − 1] kısmı elenir (hepsi ≥ a[r][c] > x); aşağı inişte r. satırın a[0..c] kısmı elenir (hepsi ≤ a[r][c] < x). r = m ya da c = −1 olunca alt matris boştur. **6 · Analiz.** Her adım r'yi artırır ya da c'yi azaltır: en çok m + n − 1 adım, O(m + n). Satır satır ikili arama O(m log n)'dir; hangisinin iyi olduğu biçime bağlıdır:

ÇIKTI · ÖRNEK 14 (TÜM X, KÂHİN: DEĞER TABLOSU)

boyut	sorgu	hatalı	merdiven en çok	m + n - 1	satır satır ikili en çok
5 x 6	30	0	10	10	15
10 x 10	59	0	15	19	40
100 x 100	679	0	117	199	700
4 x 1000	2557	0	1003	1003	40
1000 x 4	2562	0	1003	1003	3000

Kare matriste merdiven açıkça üstündür (100 × 100: en çok 117 adım, satır satır ikili 700). 4 × 1000'lik yassı matriste satır satır ikili arama (40) merdivenden (1003) çok iyidir; 1000 × 4'te ise satır satır arama 3000 adım sürer, oysa kısa kenar (4 sütun) boyunca ikili arama 4 × (⌊log₂ 1000⌋ + 1) = 40 olurdu (Alıştırma 4.11'in ipucu).

Şekil 4.10
Merdiven araması (Örnek 14): x = 15 bulunur (4 adım), x = 19 matristen çıkar (8 adım). Turkuaz hücreler incelenen elemanlar, gri hücreler elenen satır ve sütunlar



Tablo 4.8
Merdiven araması kimlik kartı

ALGORİTMA KİMLİK KARTI	**Merdiven araması (2B)** *(staircase search; saddleback search)*
Strateji	Azalt ve fethet (her adımda bir satır ya da bir sütun elenir)
Girdi önkoşulu	m × n matrisin satırları soldan sağa, sütunları yukarıdan aşağıya artan
Zaman (en iyi · ortalama · en kötü)	O(1) · O(n + m) · O(n + m)
Ek bellek	O(1)
Özellikler	Sağ üst köşeden başlar: a[r][c] > x ise sütun, < x ise satır elenir; en çok m + n − 1 adım (100 × 100: sınır 199, ölçülen en çok 117). Kare matriste satır satır ikili aramadan iyi (en çok 700 yoklama), çok yassı matriste kötü (4 × 1000: 1003'e karşı 40)
Değişmez / doğruluk	x matristeyse a[r..m−1][0..c] alt matrisindedir
Kütüphane karşılığı	C: yok · C++: yok (satır başına `std::lower_bound` yazılabilir) · Python: yok (satır başına `bisect`)
Ne zaman kullanılır	Satır ve sütunları sıralı tablolarda (ör. iki ölçüte göre sıralı ızgaralar) tek değer aramak
Ne zaman kullanılmaz	Matris yalnız satır bazında sıralıysa (her satırda ikili arama); satır ya da sütun sayısı diğerinden çok büyükse (kısa kenar boyunca ikili arama daha az adım sürer)

7 · C gerçekleştirimi. (Örnek 14). Matris satır öncelikli düz dizidedir: a[r][c] = a[r·n + c] (AP-1 H10). `size_t` indis −1 olamayacağından `c` "sütun + 1" tutulur ve `c > 0` sınanır:

c/H04_14_merdiven_aramasi.c · satır 38–54

```c
static long merdiven_ara(const int a[], size_t m, size_t n,
                         int x)
{
    size_t r = 0, c = n;        /* sağ üst köşe: (0, n - 1) */
    while (r < m && c > 0) {
        int v = a[r * n + c - 1];
        adim++;
        if (v == x) {
            return (long)(r * n + c - 1);
        } else if (v > x) {
            c--;            /* sütunu ele */
        } else {
            r++;            /* satırı ele */
        }
    }
    return -1;
}
```

8 · Üç dil karşılaştırması. C++'ta `std::vector<std::vector<int>>` ve `std::pair`; Python'da liste listesi, `(r, c)` demeti ve `c = n − 1` ile `c >= 0` (Python tamsayısı negatif olabilir):

C++ H04_14_merdiven_aramasi.cpp · satır 41–47

```cpp
static bool merdiven_ara(const Matris& a, int x,
                         std::pair<std::size_t, std::size_t>& kon)
{
    std::size_t m = a.size(), n = m > 0 ? a[0].size() : 0;
    std::size_t r = 0, c = n;   // c: sütun + 1 (size_t)
    while (r < m && c > 0) {
        int v = a[r][c - 1];
```

Python H04_14_merdiven_aramasi.py · satır 33–35

```python
    r, c = 0, n - 1             # Python'da c = -1 sorun değil
    while r < m and c >= 0:
        v = a[r][c]
```

9 · Test. Kâhin değer tablosudur (`calloc` bayt dizisi, `std::vector<char>`, `set`): her x için beş boyutta hatalı 0. **10 · Deney.** Tablo sayaçlı bir deneydir; en çok adım m + n − 1'i aşmaz. **11 · Varyantlar.** Sol alt köşeden başlamak, kısa kenar boyunca ikili arama.

5. Çözümlü Örnekler

Örnek merdiveninin on dört basamağının dokuzu Bölüm 4'te (Örnek 1–5, 7, 10, 11, 14), Örnek 6 ve 12'nin dil boyutu Bölüm 8'de işlendi. Bu bölüm kalan basamakları (6, 8, 9, 12, 13) ve üstel/interpolasyon aramasını çözer; C++ ve Python karşılıkları Ek A'dadır, dosya listesi `kodlar/KOD_DIZINI.md`'dedir.

5.1 Orta nokta taşması: int32 sınırında (lo + hi) / 2 (Örnek 6, Δ)

Bloch'un (2006) bildirdiği hatayı 2 milyar elemanlık dizi ayırmadan gösterebiliriz; yalnızca indis aritmetiği önemlidir. Örnek 6 a[i] = i olan **sanal** bir dizi kullanır. `int32_t` toplamının taşması tanımsız davranış olduğu için program taşmayı **gerçekten yapmaz**: toplamı `uint32_t` ile hesaplar (2^{32} modunda, tanımlı) ve `int32_t`'ye dönüştürür. Bu dönüşüm C17'de "uygulama tanımlı"dır (implementation-defined); GCC değeri 2^N modunda indirger (Free Software Foundation, t.y.). Böylece `(lo + hi) / 2`'nin 32 bitlik ikiye tümleyen `int` ile **tipik** sonucu tanımsız davranışa girmeden elde edilir:

C c/H04_06_orta_nokta_tasmasi.c · satır 29–39

```c
static int32_t sarmali_orta(int32_t lo, int32_t hi)
{
    uint32_t toplam = (uint32_t)lo + (uint32_t)hi;
    return (int32_t)toplam / 2;
}

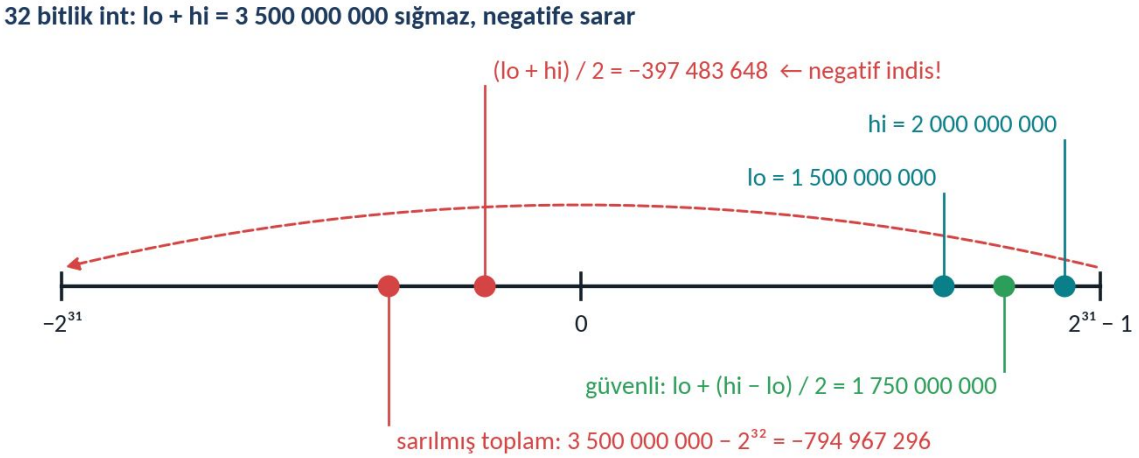
/* Güvenli: 0 <= lo <= hi iken hi - lo taşmaz, sonuç [lo, hi] içinde */
static int32_t guvenli_orta(int32_t lo, int32_t hi)
{
    return lo + (hi - lo) / 2;
}
```

ÇIKTI · C, İLK VE SON BÖLÜM (C++ ÇIKTISI AYNIDIR)

```
$ ./H04_06_orta_nokta_tasmasi
INT32_MAX = 2147483647
lo = 1500000000, hi = 2000000000
gerçek lo + hi (64 bit)      = 3500000000
(lo + hi) / 2, 32 bit sarma  = -397483648   <- negatif!
lo + (hi - lo) / 2           = 1750000000
(lo + hi) >>> 1 (işaretsiz)  = 1750000000

Hata hangi n'de başlar? (son eleman aranırken)
  n = 1073741824: en büyük lo + hi = 2147483646  <= INT32_MAX: taşmaz
  n = 1073741825: en büyük lo + hi = 2147483648  > INT32_MAX: TAŞAR
```

Şekil 4.11
32 bitlik sayı doğrusunda orta nokta taşması: lo + hi = 3 500 000 000 sığmaz, 2^{32} eksiğine (-794 967 296) sarar ve ikiye bölünce -397 483 648 olur; güvenli hesap 1 750 000 000



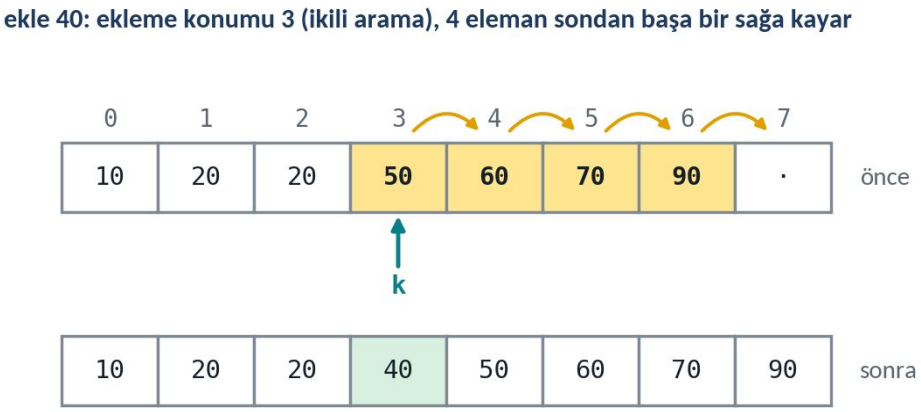
C/C++'ta işaretli taşma tanımsız davranıştır (şekil tipik sarmayı gösterir); Python'da taşma yok

İkinci adımda sarmalı orta nokta negatif bir indis üretir; program a[mid]'i okumadan durur (gerçek dizide bu, tanımsız davranış olan dizi dışı erişimdir). Güvenli orta nokta aramayı $\lfloor \log_2(2 \cdot 10^9) \rfloor + 1 = 31$ adımda bitirir. Son bölüm hatanın başladığı n'i arar: kapalı aralıkta son eleman aranırken en büyük toplam lo + hi = 2(n − 1)'dir; n = 1 073 741 824 (2^{30}) için bu $2^{31} - 2$'dir ve taşmaz, ilk taşma n = 1 073 741 825'te ($2^{30} + 1$) olur. Yarı açık sözleşmede en büyük toplam (n − 1) + n = 2n − 1'dir; ilk taşma yine n = 2^{30} + 1'dedir. Bloch'un "2^{30} ya da daha fazla elemanlı" ifadesi yuvarlatılmış bir sınırdır: kendi kodu da (high = a.length − 1) tam 2^{30} elemanda taşmaz. Python'daki öykü Bölüm 8'dedir.

5.2 Ekleme konumu ve sıralı diziye ekleme (Örnek 8)

Sıralı diziye eklerken önce yer lower_bound ile bulunur (x varsa ilk kopyanın önü), sonra a[k..n − 1] **sondan başa** bir sağa kaydırılır; baştan kaydırmak taşınmamış elemanların üzerine yazar. Örnek 8'in izlemesinde 40 için konum 3, 4 kaydırma (Şekil 4.12).

Şekil 4.12
Sıralı diziye ekleme (Örnek 8): 40 için ekleme konumu k = 3 ikili aramayla bulunur, a[3..6] sondan başa bir sağa kayar (4 kaydırma)



c/H04_08_ekleme_konumu.c · satır 40–49

```c
static size_t sirali_ekle(int a[], size_t n, int x)
{
    size_t k = ekleme_konumu(a, n, x);
    for (size_t i = n; i > k; i--) {
        a[i] = a[i - 1];
        kaydirma++;
    }
    a[k] = x;
    return k;
}
```

Arama ucuzdur (n = 4000 eklemede toplam 42 279 yoklama, n $\log_2$ n düzeyi), kaydırma pahalıdır: rastgele sırayla n ekleme ≈ $n^2/4$ kaydırma ister (Örnek 8'de kaydırma/($n^2/4$) oranı 1,014–0,988). Tek tek ekleyerek sıralı dizi kurmak $\Theta(n^2)$'dir: gelecek haftanın **araya ekleme sıralaması** (insertion sort; H5). İkili arama karşılaştırmayı azaltır, kaydırmayı azaltmaz; Mini-OBS 2 v2.3 bunu 10^5 kayıtta ölçer (Bölüm 10).

5.3 Döndürülmüş sıralı dizide arama (Örnek 9)

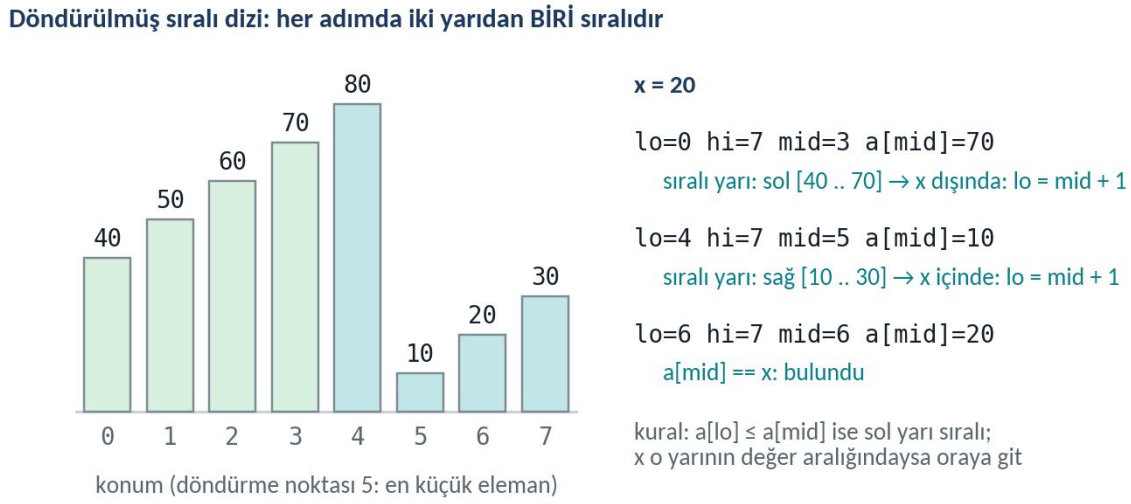
[10, 20, …, 80] üç adım sola döndürülünce [40, 50, 60, 70, 80, 10, 20, 30] olur: sıralı değildir ama iki sıralı parçadan oluşur. Değişmezi ("x varsa a[lo..hi] içinde") korumak için her adımda **hangi yarının sıralı olduğuna** bakılır: a[lo] ≤ a[mid] ise sol yarı, değilse sağ yarı sıralıdır. x sıralı yarının değer aralığındaysa oraya, değilse öbür yarıya gidilir. x = 20 için ilk adımda sıralı olan sol yarı [40 .. 70] x'i içermez ve sağa gidilir; ikinci adımda sıralı olan sağ yarı [10 .. 30] x'i içerir; üçüncü yoklamada bulunur (Şekil 4.13).

c/H04_09_dondurulmus_dizi.c · satır 19–43

```c
static long dondurulmus_ara(const int a[], size_t n, int x)
{
    long lo = 0, hi = (long)n - 1;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] == x) {
            return mid;
```

```c
        }
        if (a[lo] <= a[mid]) {              /* sol yarı a[lo..mid] sıralı */
            if (a[lo] <= x && x < a[mid]) {
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        } else {                            /* sağ yarı a[mid..hi] sıralı */
            if (a[mid] < x && x <= a[hi]) {
                lo = mid + 1;
            } else {
                hi = mid - 1;
            }
        }
    }
    return -1;
}
```

Şekil 4.13
Döndürülmüş sıralı dizi (Örnek 9): iki sıralı parça; her adımda iki yarıdan biri sıralıdır ve x'in o yarının değer aralığında olup olmadığı sınanır (x = 20)



Program m = 1 … 100 için her döndürmede ve her x için doğrusal kâhinle karşılaştırır (m = 100: 20 100 sorgu, hatalı 0; en çok 7 yoklama = $\lfloor \log_2 m \rfloor + 1$). Döndürme noktası (en küçük eleman) da ikili aramayla bulunur: elemanlar farklıysa "a[mid] > a[hi]" yüklemi monotondur (Alıştırma 4.5). **Tekrar eden elemanlar** yöntemi bozar: a[lo] = a[mid] = a[hi] iken hangi yarının sıralı olduğu anlaşılamaz. `dondurulmus_ara_tekrarli` bu durumda iki ucu birer daraltır; a[lo] ve a[hi], x olmayan a[mid]'e eşit olduğundan değişmez korunur:

c/H04_09_dondurulmus_dizi.c · satır 73–76

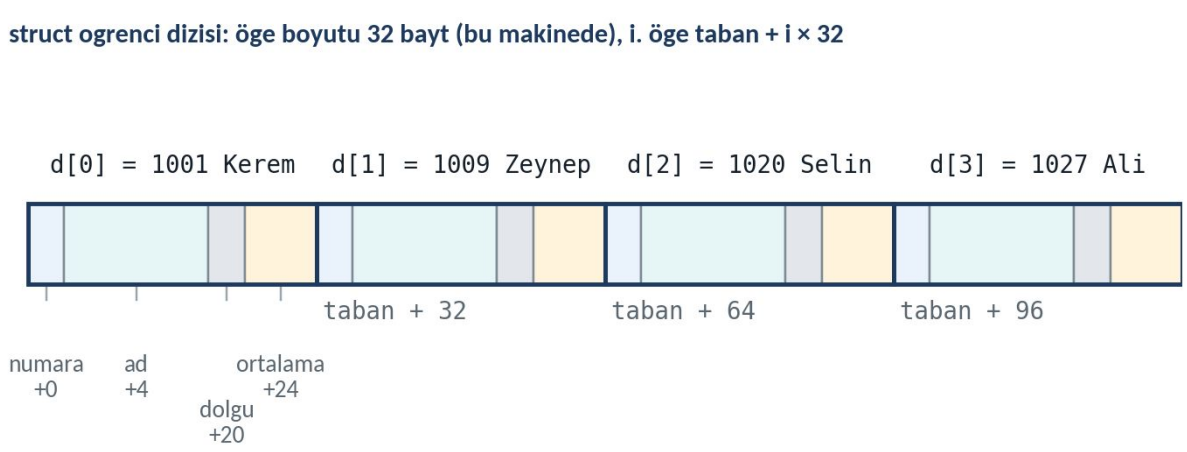
```c
        if (a[lo] == a[mid] && a[mid] == a[hi]) {
            lo++;
            hi--;
        } else if (a[lo] <= a[mid]) {
```

Bunun bedeli en kötü durumda O(n)'dir: 999 tane 2 ve tek bir 1 içeren dizide 1'i bulmak, 1'in konumuna göre 9–501 yoklama sürer; hepsi 2 olan 1 000 elemanlı dizide x = 3 araması 500 yoklamadan sonra −1 döndürür. Python'da koşul zincirleme yazılır (`a[lo] <= x < a[mid]`); C'de bu yazım derlenir ama anlamı farklıdır (Bölüm 8).

5.4 Genel arama: karşılaştırıcı işaretçisi ve bsearch (Örnek 12)

Kayıt dizisinde (numara, ad, ortalama) numaraya da ada da göre aramak için **karşılaştırma ölçütünü parametre** yaparız. C'de bu bir fonksiyon işaretçisidir (AP-1 H12): `int (*kars)(const void *, const void *)`; a < b ise negatif, eşitse 0, a > b ise pozitif. Fonksiyon ögenin türünü bilmediğinden diziyi `void *` taban, öge sayısı ve öge **boyutu** ile alır ve mid. ögenin adresini bayt aritmetiğiyle hesaplar (Bölüm 3.5; Şekil 4.14). Parametre sırası `bsearch` ile aynıdır.

Şekil 4.14
struct ogrenci dizisinin bellek düzeni (bu makinede öge boyutu 32 bayt; Kavram 10): mid. öge taban + mid × boyut baytta başlar; ögenin içinde numara, ad, dolgu ve ortalama alanları



c/H04_12_genel_arama.c · satır 40–62

```c
static long genel_ara(const void *anahtar,
                      const void *taban, size_t n,
                      size_t boyut,
                      int (*kars)(const void *,
                                  const void *))
{
    size_t lo = 0, hi = n;
    while (lo < hi) {
        size_t mid = lo + (hi - lo) / 2;
        const char *oge = (const char *)taban
                          + mid * boyut;
        int s = kars(anahtar, oge);
        yoklama++;
        if (s == 0) {
            return (long)mid;
        } else if (s > 0) {
            lo = mid + 1;   /* anahtar > öge */
        } else {
            hi = mid;       /* anahtar < öge */
        }
```

```
60      }
61      return -1;
62  }
```

ÇIKTI · ÖRNEK 12 (ARAMA BÖLÜMLERİNDEN SEÇME SATIRLAR)

```
Numaraya göre arama
  numara 1027: konum 5 -> 1027 Ali    3.90 (1 yoklama)
  numara 1030: bulunamadı (4 yoklama)
Ada göre arama (aynı genel_ara, başka karşılaştırıcı)
  ad Selin : konum 7 -> 1020 Selin  3.05 (3 yoklama)
  ad Ayse  : bulunamadı (3 yoklama)
Önek araması: "E" ile başlayan adlar (lower_bound + tarama)
  lower_bound konum 4 (4 yoklama); eşleşenler: Ece Emre
kâhin (bsearch / std::lower_bound / bisect) ile aynı mı? evet
```

Aynı `genel_ara` iki karşılaştırıcıyla iki farklı sıradaki diziyi arar; **ön koşul** da aynı karşılaştırıcıya göredir (numaraya göre sıralı dizi ada göre sıralı değildir; program ada göre sıralı ayrı bir kopya tutar ve ikisini `sirali_mi` ile denetler). `numara_kars` `x - y` yerine `(x > y) - (x < y)` döndürür: fark `int`'e sığmayabilir (Hata Avcısı 5). `genel_alt_sinir` ile **önek araması** yapılır: "E" ile başlayan ilk ad lower_bound'la bulunur, sonra önek uydukça ilerlenir (Mini-OBS 2 v2.3'ün soyad araması). `strcmp` bayt sırasıyla karşılaştırır; bu, Türkçe alfabe sırası değildir (H5). Kâhinler: `bsearch`, `std::lower_bound`, `bisect_left(..., key=...)`.

5.5 Doğrusal ve ikili arama: sayaç ve süre (Örnek 13)

Örnek 13, Haftanın Deneyi'nin taşınabilir küçük sürümüdür: n = 8 … 4096 için yarısı başarılı 1000 sorgu üretir, sonuçları karşılaştırır ve sayaçları kuramla yan yana yazar. %50/%50 karışımda doğrusal aramanın beklenen karşılaştırması ½·(n + 1)/2 + ½·n = (3n + 1)/4'tür:

ÇIKTI · ÖRNEK 13 (SEÇME SATIRLAR)

```
$ ./H04_13_sure_karsilastirma
     n  doğrusal ort.  (3n+1)/4  ikili ort.  ikili en çok  ⌊log2 n⌋+1  aynı
     8          6.25      6.25        2.94             4           4  evet
    64         48.04     48.25        5.60             7           7  evet
   512        386.66    384.25        8.53            10          10  evet
  1024        773.22    768.25        9.52            10          11  evet
  4096       3076.20   3072.25       11.52            12          13  evet
```

n = 1024'te bu sorgularda en çok 10 yoklama görüldü, sınır 11'dir: $\lfloor \log_2 n \rfloor + 1$ bir üst sınırdır, her örneklemde erişilmez. Ölçülen fonksiyonlar sayaçsızdır (sayaç artırımı da süreye girer). `--olc` ile süreler ve kesişim kestirimi yalnızca stderr'e yazılır (stdout üç dilde aynı kalsın diye); tek çalıştırmada C kesişimi ≈ n = 77 (64–128). Bölüm 6 aynı soruyu sıkı bir ölçüm düzeniyle yanıtlar.

5.6 Kısa bakış: üstel arama ve interpolasyon araması

Üstel arama (exponential/galloping search), boyut bilinmediğinde ya da x başa yakınken kullanılır: sınır 1, 2, 4, … ikiye katlanarak a[sınır − 1] ≥ x olan ilk sınır bulunur, sonra yalnızca [sınır/2, sınır) içinde ikili arama yapılır. Konum k ise toplam O(log k), n'den bağımsız. Kavram 4'te n = 10^6 için k = 5'te 7, k = 100'de 14 yoklama (tüm dizide ikili 20); k = 999 999'da ise 39 yoklamayla ikiliden kötüdür.

İnterpolasyon araması (interpolation search), rehberde "S"yi sonlarda açmaya benzer: mid yerine x'in değerine göre kestirilen konum lo + (x − a[lo])·(hi − lo)/(a[hi] − a[lo]) yoklanır. Düzgün dağılımda kuramsal ortalama log log n mertebesindedir (Knuth, 1998); çarpık dağılımda kestirim sürekli yanılır. Kavram 5'te n = 10^5, 1000 başarılı aramada ortalama yoklama: düzgün dizide interpolasyon 3,25, ikili 15,75; a[i] = i^3 dizisinde interpolasyon 114,41, ikili 15,75. İkili aramanın güvencesi dağılımdan bağımsızdır; interpolasyonun kazancı bir **varsayıma** dayanır.

6. Haftanın Deneyi: Doğrusal ve İkili Arama, Kesişim Noktası

Soru. Doğrusal arama küçük dizilerde gerçekten daha mı hızlıdır? İkili arama hangi n'den sonra öne geçer, bu kesişim C, C++ ve Python'da nasıl değişir? **Düzen.** Deney programı (`kodlar/deney/` altında `c/`, `cpp/` ve `python/` klasörlerinde `H04_deney_arama`) Örnek 1, 2 ve 3'ün sayaçsız fonksiyonlarını ölçer. Veri a[i] = 2i + 1; sorgular ortak LCG ile üretilir, yarısı başarılı (tek), yarısı başarısızdır (çift). n = 2 … 10^7 arasında 33 değer; C ve C++ -O2, C ayrıca -O0. Her ölçüm 1 ısınma + 5 tekrarın ortancasıdır (Sedgewick & Wayne, 2011); plan üç kez geçildi ve her noktada geçişlerin en küçük ortancası alındı. Süre **işlemci zamanıdır** (`clock()`, `time.process_time`). Ortam (`deney/OLCUM_ORTAMI.txt`): Intel(R) Xeon(R) Processor @ 2.80GHz, 2 çekirdekli paylaşımlı sanal makine, gcc/g++ 13.3.0, Python 3.11.15; toplam deney süresi 259,5 s.

Tablo 4.9

*Arama başına ortanca süre (işlemci zamanı; C ve C++ -O2 aksi yazılmadıkça; h04_deney_olcum.csv). * kestirim (h04_deney_kestirim.csv)*

n	C doğ.	C nöb.	C ikili	C++ doğ.	C++ ikili	C -O0 doğ.	C -O0 ikili	Py doğ.	Py nöb.	Py ikili
2	4,5 ns	4,1 ns	6,9 ns	4,8 ns	9,5 ns	9,3 ns	18,0 ns	214,9 ns	142,0 ns	186,0 ns
10	10,2 ns	8,7 ns	16,5 ns	10,9 ns	19,9 ns	23,6 ns	36,3 ns	351,8 ns	277,7 ns	314,4 ns
20	15,0 ns	11,4 ns	21,9 ns	16,6 ns	24,8 ns	47,6 ns	55,7 ns	541,1 ns	481,6 ns	386,8 ns
50	32,4 ns	21,5 ns	29,2 ns	32,5 ns	34,1 ns	105,6 ns	62,6 ns	1,08 µs	994,4 ns	500,5 ns
100	56,8 ns	35,1 ns	33,9 ns	57,0 ns	39,6 ns	193,9 ns	71,5 ns	1,99 µs	1,83 µs	607,8 ns
10^3	466,4 ns	269,6 ns	53,6 ns	469,1 ns	62,9 ns	1,90 µs	112,9 ns	22,5 µs	21,2 µs	1,02 µs
10^4	4,70 µs	2,37 µs	72,4 ns	4,62 µs	82,9 ns	18,6 µs	148,4 ns	229 µs	222 µs	1,31 µs
10^5	47,5 µs	23,3 µs	98,2 ns	46,9 µs	116,1 ns	190 µs	220,5 ns	2,1 ms	2,1 ms	2,34 µs
10^6	512 µs	266 µs	197,0 ns	565 µs	216,3 ns	2,1 ms	461,4 ns	23,2 ms	23,3 ms	4,29 µs
10^7	6,1 ms	5,1 ms	566,3 ns	6,3 ms	584,1 ns	19,3 ms	757,3 ns	≈ 225 ms*	≈ 224 ms*	7,36 µs

Kuramla karşılaştırma ve yorum.

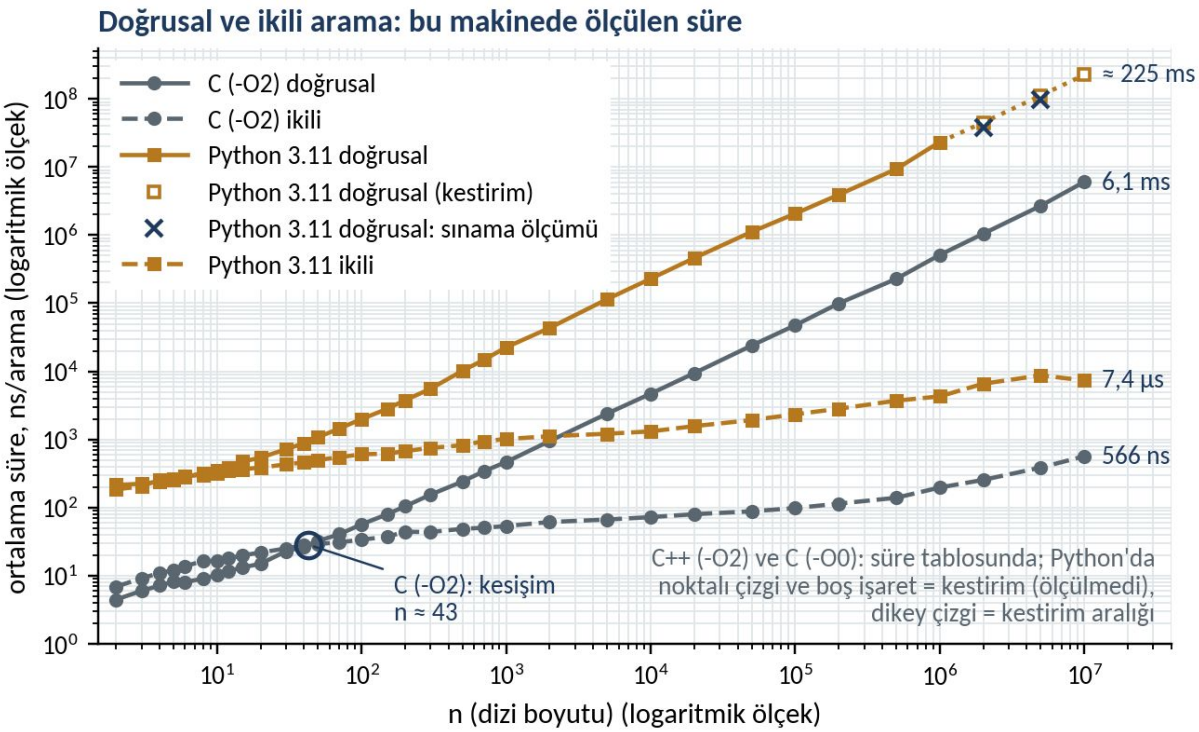
- **Küçük dizide doğrusal arama kazanır (derlenen dillerde).** n ≤ 20'de doğrusal/ikili süre oranı C -O2'de 0,55–0,69, C++ -O2'de 0,50–0,67 oldu; oysa n = 20'de doğrusal arama 15,2, ikili arama yalnızca 4,1 karşılaştırma yapar. Kalıcı kesişim bu makinede, bu çalıştırmada C -O2'de n ≈ 43 (40–50), C++ -O2'de ≈ 59, C -O0'da ≈ 26; nöbetçiliyle C -O2'de ≈ 94. Önceki tam çalıştırma birkaç komşu n kaydı (Tablo 4.10): doğru ifade **"birkaç on eleman"**dır. -O0'da kesişimin küçülmesi döngü başına sabit maliyetin büyümesiyle uyumludur.
- **Python'da düz doğrusal aramanın küçük-n üstünlüğü gürültü düzeyini aşmadı.** Deneyde n ≤ 8'de doğrusal/ikili oranı 0,93–1,16 arasında gidip geldi (kalıcı geçiş n ≈ 7, ilk geçiş n = 2; önceki çalıştırmada n = 2'de bile ikili hızlıydı). Ayrı denetimde (5 tur × 20 000 sorgu, en küçük değerler) oran n ≤ 8'de 1,005–1,110 çıktı: **hiçbir n'de doğrusal arama ikiliden hızlı değildi** (fark en çok %11). Nöbetçili sürüm ikiliden n ≈ 14'e (12–15) kadar hızlıydı. Olası açıklama (ölçülmedi): süreyi

yürütülen bayt kodu belirler; `for i in range(n)` her çağrıda bir yineleyici kurar, `x in a` ise döngüyü C'de çalıştırır: Python'da kesişim gerçekleştirime bağlıdır.

- **Nöbetçinin kazancı dile bağlıdır.** Karşılaştırma 2n + 1'den n + 2'ye iner; süre oranı (nöbetçili/klasik) n = 10^3 … 10^6'da C -O2'de 0,49–0,58, Python'da 0,94–1,03.
- **Dil ve derleme farkları.** Python/C -O2 oranı doğrusal aramada 43,5–48,8 kat (n = 10^3 … 10^6), ikili aramada 13,0–23,9 kat (n = 10 … 10^7); -O0 C'yi doğrusal aramada 4,0–4,1 kat yavaşlattı. C++ -O2 ile C -O2 aynı mertebededir.
- **Büyüme hızları kuramı izler, ama önbellekle.** Doğrusal aramanın log–log eğimi 10^4 … 10^6'da C -O2'de 1,02, Python'da 1,00 (Θ(n) için 1). İkili aramanın C -O2 eğimi 10^3 … 10^5'te 0,13, 10^3 … 10^7'de 0,26: log n yavaş büyür, ama dizi önbellekten taşınca yoklama başına süre artar (Şekil 4.15).
- **Sayaçlar kuramla birebir (üç dilde aynı).** İkili aramada en çok yoklama 33/33 n'de $\lfloor \log_2 n \rfloor + 1$'e eşittir, hiçbir n'de aşılmaz; n = 10^6'da ortalama başarılı 18,98, başarısız 19,95, en çok 20. Doğrusal aramanın ortalaması (3n + 1)/4'ü izler (n = 10^6: 748 998, kuram 750 000).
- **Python'da büyük n: kestirim.** Python doğrusal arama n ≤ 10^6 ölçüldü; ötesi için t ≈ c·K(n) (K: arama başına karşılaştırma; c = 29,9 ns, n ≥ 10^4 ölçümlerinin ortancası). n = 10^7 için kestirim ≈ 225 ms (aralık 187 ms–234 ms); aynı n'de ölçülen ikili arama 7,36 µs, oran ≈ 31 000. Model 4 ek ölçümle sınandı; sapma en çok %18,8 oldu (kestirim, büyüklük sırası için yeterli).

Şekil 4.15

Arama başına süre (log–log), C -O2 ve Python: düz çizgi doğrusal, kesikli ikili; noktalı = kestirim, × = sınama



Sınırlılıklar. Ölçümler 2 çekirdekli paylaşımlı bir sanal makinede yapıldı. Bir noktanın üç geçişteki ortancaları arasındaki oran (en büyük/en küçük) ortanca 1,31 oldu; 135/390 noktada 1,5'i aştı, en kötüsünde 4,3 oldu; art arda sekiz çalıştırmada ortancalar en çok 1,23 kat değişti. Bu yüzden geçişlerin en küçük ortancası kullanıldı (yük yalnızca yavaşlatır) ve kesişimler aralıkla verildi (H1: "tek ölçüm yetmez"). `clock()`'un birimi 1 µs'dir; tekrar başına en kısa süre bundan birkaç yüz kat uzundur. Dal tahmini ve önbellek açıklamaları performans sayaçlarıyla ölçülmedi. **Kendi makinenizde**

`kodlar/deney/` altında `python3 olcum_calistir.py` ile tekrarlayın: sayaçlar aynı, süreler farklı çıkacaktır.

Tablo 4.10
Kesişim noktaları: oran = t(doğrusal)/t(ikili); kalıcı kesişim: sonraki tüm n'lerde oran ≥ 1

Dil	Derleme	Karşılaştırma	Kalıcı kesişim n*	Komşu ölçümler (oran)	Önceki tam çalıştırma
C	-O2	doğrusal/ikili	≈ 43	40 (0,96) – 50 (1,11)	≈ 39
C++	-O2	doğrusal/ikili	≈ 59	50 (0,95) – 70 (1,05)	≈ 58
C	-O0	doğrusal/ikili	≈ 26	20 (0,86) – 30 (1,08)	≈ 21
Python	-	doğrusal/ikili	≈ 7	6 (0,97) – 8 (1,06)	n = 2'de bile ikili hızlı
C	-O2	nöbetçili/ikili	≈ 94	70 (0,84) – 100 (1,04)	≈ 89
C++	-O2	nöbetçili/ikili	≈ 116	100 (0,92) – 150 (1,16)	≈ 123
C	-O0	nöbetçili/ikili	≈ 32	30 (0,91) – 40 (1,30)	≈ 29
Python	-	nöbetçili/ikili	≈ 14	12 (0,89) – 15 (1,07)	≈ 13

7. Hata Avcısı

Dokuz hatanın her biri `kodlar/hata_avcisi/` altında kasıtlı hatalı ve düzeltilmiş dosya çifti olarak durur; belirtiler bu makinede gözlenen çıktılardır (`hata_avcisi/ciktilar/`). Hiçbiri derleyici uyarısı üretmez: ikili aramanın hataları mantık hatası ya da tanımsız davranıştır. Sonsuz döngüler varsayılan çalıştırmada bir adım sınırıyla korunur; gerçek belirti `korumasiz` argümanıyla görülür.

Tablo 4.11
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Kapalı aralıkta `lo = mid`	ara(50): “20 adımda bitmedi: lo = 3, hi = 4 değişmiyor”; korumasız: zaman aşımı	lo == mid iken aralık daralmaz	`lo = mid + 1`, `hi = mid - 1`: aralık her adımda küçülmeli
2	Bir-fark: `hi = n` ile `<=`; `hi = n - 1` ile `<`	(a) dizi dışı okuma (ASan: heap-buffer-overflow); (b) son eleman bulunamaz	Sözleşmenin dört ögesi karışmış	Tek sözleşme seç; n = 0, 1, ilk, son, dış değerlerle sına
3	Orta nokta `(lo + hi) / 2` (int)	Negatif mid; 40 adımda bitmez; UBSan: signed integer overflow	İşaretli taşma: tanımsız davranış	`lo + (hi - lo) / 2`
4	Sırasız diziye ikili arama	dizideki 7 elemandan 3 tanesi bulunabildi	Ön koşul bozuk	Önce sırala (H5) ya da doğrusal ara; ön koşulu geliştirmede denetle
5	Karşılaştırıcıda `return x - y`	`bsearch` iki elemanı bulamaz; UBSan: signed integer overflow	Fark `int`'e sığmaz (UB)	`(x > y) - (x < y)`
6	Python `list.index` bulamayınca	`ValueError: 6 is not in list`; sonraki sorgular yapılmaz	`index` istisna fırlatır	`x in a`, `try/except ValueError` ya da `None` döndüren fonksiyon
7	Son konum = `lower_bound − 1` (C++)	x = 2: ilk 1, son 0, adet 0; x = 1'de son = −1	lower_bound − 1, x'ten önceki elemandır	`upper_bound − 1`; önce varlık denetimi; `equal_range`

#	Hata	Belirti (bu makinede)	Neden	Çözüm
8	Kayan noktada `while f(m) != 0.0`	53. yinelemede m değişmez; korumasız: zaman aşımı	√2 double'da yok; f(m) hiç tam sıfır olmaz	Tolerans + en çok yineleme + m == a / b denetimi
9	`size_t` ile kapalı aralık	hi = SIZE_MAX, mid ≈ 9,2·10^{18}; korumasız: SIGSEGV (139)	`hi = mid - 1` mid = 0'da sarar; a[mid] dizi dışı (UB)	`size_t` ile yarı açık [0, n) ya da işaretli indis

HA1 · Sonsuz döngü (`lo = mid`). a = [10, 20, 30, 40, 50]'de ara(50): lo = 3, hi = 4 iken mid = 3 ve `lo = mid` hiçbir şeyi değiştirmez. Kural: **her güncelleme mid'i aday bölgeden çıkarmalıdır.**

C `hata_avcisi/H04_HA1_sonsuz_dongu.c · satır 36–39`

```c
        } else if (a[mid] < x) {
            lo = mid;               /* HATA: mid zaten elendi; mid + 1 */
        } else {
            hi = mid - 1;
```

HA2 · Bir-fark. (a) `hi = n` ile `lo <= hi`: a[n] okunur (dizi dışı erişim, tanımsız davranış; AddressSanitizer yakalar; varsayılan çalıştırmada sonuç yalnızca yanlıştır: `ara(999) = 5`). (b) `hi = n - 1` ile `lo < hi`: lo == hi olan son aday incelenmez. Hata, iki doğru sözleşmenin **ögelerini karıştırmaktır** (Tablo 4.1).

C `hata_avcisi/H04_HA2_bir_fark.c · satır 32–33`

```c
    long lo = 0, hi = n;                /* HATA: kapalı aralıkta n - 1 */
    while (lo <= hi) {
```

HA3 · Orta nokta taşması. n = 2·10^9 (sanal dizi), son eleman aranırken lo + hi = 2 999 999 999 > INT_MAX. Bu derleyicide, bu çalıştırmada değer sarar ve mid negatif çıkar (-O2 ve -O0'da aynı); tanımsız davranışta bu bir güvence değildir (ISO/IEC, 2018).

C `hata_avcisi/H04_HA3_orta_nokta_tasmasi.c · satır 31–31`

```c
        int mid = (lo + hi) / 2;        /* HATA: lo + hi taşabilir (UB) */
```

HA4 · Sırasız dizi. Uyarı da çökme de yoktur: doğruluk kanıtı "dizi sıralı" ön koşuluna dayanır. Düzeltilmiş sürüm önce `sirali_mi` ile denetler (O(n): yalnızca geliştirmede ya da dış veride).

C `hata_avcisi/H04_HA4_sirasiz_dizi.c · satır 30–30`

```c
    static const int a[] = {9, 2, 7, 4, 1, 8, 3};   /* HATA: sırasız */
```

HA5 · Karşılaştırıcı taşması (`return x - y`). 2 000 000 000 − (−1 800 000 000) `int`'e sığmaz; bu çalıştırmada sonuç negatif çıkar ve `bsearch` yanlış yarıya gider: orta nokta taşmasının karşılaştırıcıdaki ikizi. `(x > y) - (x < y)` hep −1, 0 ya da 1'dir.

C `hata_avcisi/H04_HA5_karsilastirici_tasmasi.c · satır 15–19`

```c
static int kars(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return x - y;               /* HATA: x - y taşabilir (UB) */
}
```

HA6 · ValueError (`list.index`). "Bulamazsa −1 döner" sanılır; `index` istisna fırlatır ve program ilk bulunamayan değerde durur. Düzeltilmiş dosya: `x in a`, `try/except ValueError`, `None` döndüren fonksiyon.

Python **hata_avcisi/H04_HA6_index_valueerror.py · satır 10–13**

```python
for x in [9, 3, 6, 5]:
    k = a.index(x)              # HATA: 6 yok -> ValueError
    print(f"a.index({x}) = {k}")
print("tüm sorgular bitti")
```

HA7 · Son konum hatası (C++, `lower_bound − 1`). lower_bound − 1, **x'ten önceki** elemanın konumudur (x ilk elemansa −1, dizi dışı). Son konum upper_bound − 1'dir ve yalnızca lower ≠ upper iken anlamlıdır; düzeltilmiş sürüm `std::equal_range` kullanır.

C++ **hata_avcisi/H04_HA7_alt_ust_karisikligi.cpp · satır 19–22**

```cpp
        long ilk = std::lower_bound(a.begin(), a.end(), x) - a.begin();
        long son = std::lower_bound(a.begin(), a.end(), x) - a.begin() - 1;
        //         HATA: ^^^^^^^^^^^ son konum için upper_bound gerekir
        long adet = son - ilk + 1;
```

HA8 · Kayan noktada tam eşitlik (Python). √2 double olarak yazılamaz; ona en yakın iki double'da $f = \mp 4{,}44 \cdot 10^{-16}$'dır. 53. yinelemeden sonra m değişmez, döngü sonsuza dek döner: Örnek 11'in üç durma koşulu bu yüzden vardır ($x^3 - x - 2$'de tam sıfır çıkması bir rastlantıdır).

Python **hata_avcisi/H04_HA8_kayan_esitlik.py · satır 28–28**

```python
    while f(m) != 0.0:              # HATA: kayan noktada tam eşitlik
```

HA9 · İşaretsiz sarma (`size_t` ile kapalı aralık). İşaretsiz aritmetik tanımlıdır: mid = 0 iken `hi = mid - 1` SIZE_MAX (18 446 744 073 709 551 615) olur; sonraki adımda mid ≈ $9{,}2 \cdot 10^{18}$ ile a[mid] okunur (dizi dışı, tanımsız davranış; bu çalıştırmada SIGSEGV). Boş dizide `hi = n - 1` de sarar. Çözüm: `size_t` ile yarı açık sözleşme ya da işaretli indis (Örnek 3).

C **hata_avcisi/H04_HA9_size_t_sarmasi.c · satır 22–23**

```c
    size_t lo = 0, hi = n - 1;          /* HATA: n = 0 iken SIZE_MAX */
    while (lo <= hi) {
```

8. Dil Penceresi: C · C++ · Python

Tablo 4.12
Bu haftanın dil farkları

Öğe	C	C++	Python
"Bulunamadı"	`-1`	`-1`; `std::find` → `end()`	`None`; `a.index` → `ValueError`
Orta nokta	`lo + (hi - lo) / 2`	aynı (C++20: `std::midpoint`)	`(lo + hi) // 2` (taşma yok)
İşaretli taşma	tanımsız davranış	tanımsız davranış	yok (keyfi duyarlık)
İşaretsiz taşma	2^N modunda (tanımlı)	aynı	yok; `& 0xFFFFFFFF` ile taklit
Kütüphane: var mı?	`bsearch` → işaretçi ya da `NULL`	`std::binary_search` → `bool`	`x in a` (doğrusal; C'de çalışır)

Öğe	C	C++	Python
Kütüphane: konum	yok (elle yazılır)	`lower_bound`, `upper_bound`, `equal_range`	`bisect_left`, `bisect_right`
Karşılaştırıcı	`int (*)(const void *, const void *)`: < 0 / 0 / > 0	"küçüktür" yüklemi (katı zayıf sıralama)	`key=` fonksiyonu (bisect: 3.10+)
Fonksiyonu değer olarak	fonksiyon işaretçisi	şablon + lambda	birinci sınıf fonksiyon, `lambda`
assert	`<assert.h>`; `-DNDEBUG` kapatır	`<cassert>`	deyim; `python3 -O` kapatır
Zincir karşılaştırma	`a <= x && x < b`	aynı	`a <= x < b`
İndis türü	`size_t` (işaretsiz)	`std::size_t`	`int`; `a[-1]` geçerli (son eleman)

Orta nokta üç dilde (Örnek 6, Δ). C ve C++ aynı çıktıyı verir. Python'da taşma yoktur; 32 bitlik sarma ancak `& 0xFFFFFFFF` ile **taklit** edilir ve C'nin sıfıra doğru kesen bölmesi için `//` yerine ayrı bir fonksiyon gerekir (`-7 // 2` Python'da −4, C'de `-7 / 2` −3; H1). C++ ve Python kesitleri (`sifira_dogru_bol`, `sarmali_orta`) ve üç dilin çıktıları Ek A.6'dadır. Python çıktısında `(lo + hi) // 2 (Python) = 1750000000` doğru sonuçtur; 32 bitlik taklit (`(lo + hi) / 2, 32 bit taklit = -397483648`) C'nin tipik sonucunu verir.

Kütüphane karşılıkları. Algoritmayı kendimiz yazdık; kütüphaneler onun **kâhinidir** (Python Software Foundation, t.y.). C'de tek araç `bsearch`'tür (Kavram 1): işaretçi döndürür (konum `p - a`), bulamazsa `NULL`; eşit anahtarlardan hangisini döndüreceği belirtilmemiştir. a = [1, 2, 2, 2, 4, 4, 7, 9, 9, 9, 9, 12] için bu makinede (glibc) `bsearch(9): konum 9`, `bsearch(2): konum 3`: ikisi de ilk konum değildir.

C `kavram/H04_K01_bsearch.c · satır 26–28`

```c
        const int *p = bsearch(&x, a, n, sizeof a[0], int_kars);
        if (p != NULL) {
            printf("bsearch(%d): konum %td\n", x, p - a);
```

C++'ta `<algorithm>` ailenin tamamını verir (Kavram 2): `lower_bound`/`upper_bound` **yineleyici** döndürür (konum `it - a.begin()`), `equal_range` ikisini birden, `binary_search` var/yok bilgisini (`x = 9: lower 7, upper 11, adet 4, equal_range 7..11, binary_search true`). Ölçüt bir "küçüktür" yüklemidir: azalan dizide `std::greater<int>()` verilir (`azalan dizide lower_bound(7): 3`). `std::partition_point` monoton yüklemde ilk "yanlış"ı bulur, "ilk D"nin C++ karşılığıdır (`ilk r * r > 40: r = 7`):

C++ `kavram/H04_K02_lower_bound.cpp · satır 31–37`

```cpp
    auto k = std::lower_bound(b.begin(), b.end(), 7, std::greater<int>());
    std::cout << "azalan dizide lower_bound(7): " << (k - b.begin())
              << "\n";
    // partition_point: r * r <= 40 doğru olduğu sürece sol taraftayız
    const std::vector<int> r = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    auto p = std::partition_point(r.begin(), r.end(),
                                  [](int v) { return v * v <= 40; });
```

Python'un `bisect` modülü doğrudan indis döndürür (Kavram 3: `x = 9: left 7, right 11, adet 4`); `insort` sıralı eklemeyi yapar, 3.10'da eklenen `key=` ile kayıtlar bir alana göre aranır (`key ile 1009: 1 Zeynep`). `list.index` doğrusal arar ve bulamazsa `ValueError` fırlatır (Hata Avcısı 6):

Python **kavram/H04_K03_bisect.py · satır 18–20**

```python
    kayitlar = [(1001, "Kerem"), (1009, "Zeynep"), (1027, "Ali")]
    k = bisect.bisect_left(kayitlar, 1009, key=lambda r: r[0])
    print("key ile 1009:", k, kayitlar[k][1])
```

Karşılaştırıcı sözleşmeleri (Örnek 12). C'de fonksiyon işaretçisi ve `void *` (tür bilgisi yok, boyut elle verilir); C++'ta şablon: tür ve yüklem derleme zamanında bağlanır, yüklem bir lambda olabilir; Python'da `key` fonksiyonu kayıttan anahtarı çıkarır (C++ ve Python `genel_ara`: Ek A.12).

C++'ın "küçüktür" yüklemi **katı zayıf sıralama** (strict weak ordering) olmalıdır: `kucuk(a, a)` yanlış, `kucuk(a, b)` doğruysa `kucuk(b, a)` yanlış; eşitlik `!kucuk(a, b) && !kucuk(b, a)` ile anlaşılır. `<=` gibi katı olmayan yüklem lower_bound'u upper_bound'a çevirir. C karşılaştırıcısında üç yanıt tek `int`'te döner ve taşmamalıdır (Hata Avcısı 5). **Zincir karşılaştırma:** Python'da `a[lo] <= x < a[mid]` matematikteki anlamıyla çalışır (Örnek 9); C'de önce `a[lo] <= x` (0 ya da 1) hesaplanır ve sonuç `a[mid]` ile karşılaştırılır.

9. Mühendis Gözüyle

git bisect: hata ayıklamada ikili arama. 1.0 sürümü çalışıyordu, bugünkü çalışmıyor; arada 1000 kayıt (commit) var. Hangisi hatayı getirdi? "Bu sürümde hata var mı?" yüklemi zaman içinde monotondur (hata bir kez girdiyse sonra da vardır): iyi … iyi kötü … kötü. İlk kötü sürüm cevap üzerinde ikili aramayla en çok $\lceil \log_2 1000 \rceil = 10$ testte bulunur. `git bisect` bunu yapar (Git, t.y.): kullanıcı bir iyi ve bir kötü sürüm verir, git ortadakini çıkarır, kullanıcı sınayıp "iyi/kötü" der, git aralığı yarılar.

KABUK OTURUMU (KOMUTLAR)

```
$ git bisect start
$ git bisect bad                 # şu anki sürüm hatalı
$ git bisect good v1.0           # bu etiketteki sürüm sağlamdı
  … git ortadaki bir sürümü çıkarır; derleyip sınayın:
$ git bisect good                # ya da: git bisect bad
  … ilk kötü kayıt bulunana kadar yinelenir
$ git bisect reset               # başlangıçtaki dala dön
```

Kavram 8 aynı aramayı Python'da benzetir; yüklem fonksiyon olarak aktarılır. 1000 sürümde ilk kötü 731 iken testler sırasıyla 500İ, 750K, 625İ, 688İ, 719İ, 735K, 727İ, 731K, 729İ, 730İ (İ: iyi, K: kötü); `k = 1000000: ilk kötü sürüm 765432, 20 test (⌈log2 k⌉ = 20)`:

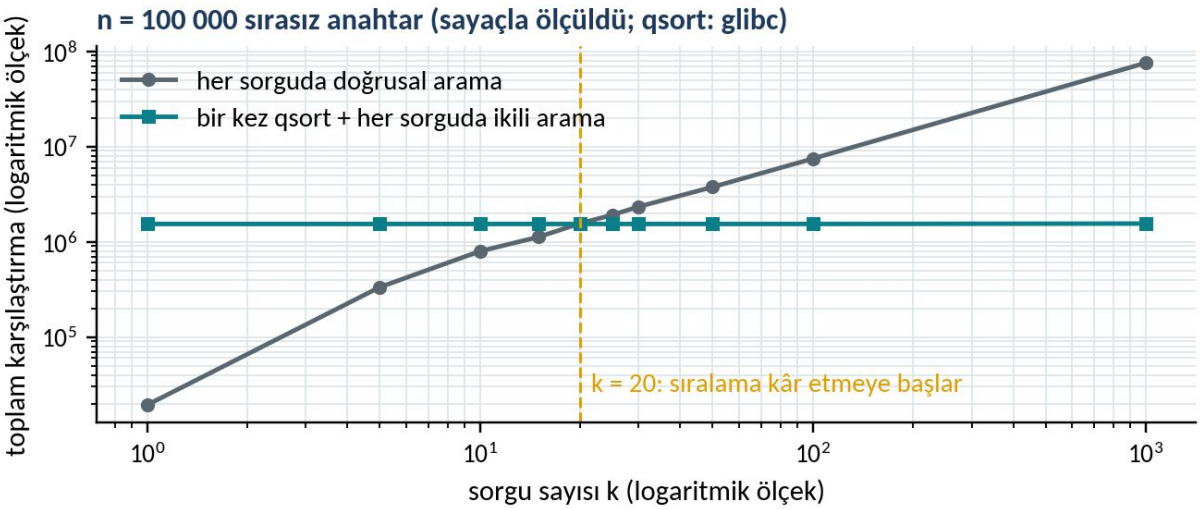
Python **kavram/H04_K08_ilk_kotu_surum.py · satır 9–18**

```python
def ilk_dogru(lo, hi, yuklem):
    """[lo, hi] içinde yuklem(s) doğru olan en küçük s (yuklem(hi) doğru).
    Değişmez: yuklem(lo - 1) yanlış, yuklem(hi) doğru."""
    while lo < hi:
        mid = (lo + hi) // 2
        if yuklem(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Ön işleme ile sorgu maliyeti ödünleşimi. n sırasız anahtarda k sorgu için: her sorguda doğrusal arama (≈ k·n/2 karşılaştırma, başarılı sorguda) ya da bir kez sırala, sonra ikili arama (≈ $n \log_2 n + k \cdot \log_2 n$).

Kavram 9 bunu **sayaçla** ölçer; sıralama için `qsort` sayaçlı bir karşılaştırıcıyla kullanılır (sıralamayı H5–H7'de kendimiz yazacağız). n = 100 000 için `qsort` (glibc) 1 536 285 karşılaştırma yaptı; n $\log_2$ n ≈ 1 660 964.

Şekil 4.16
Ön işleme ödünleşimi (Kavram 9): toplam karşılaştırma, k sorgu için doğrusal arama ile "qsort + ikili arama" (log–log). Bu girdide sıralama k = 20 sorgudan sonra kâr eder



Bu girdide sıralama k = 20 sorgudan itibaren kâr eder; 1000 sorguda doğrusal strateji 75 558 689, sıralı strateji 1 552 472 karşılaştırma yapar (≈ 49 kat). Karar verinin ne sıklıkla değiştiğine de bağlıdır: sıralı dizide her ekleme O(n) kaydırmadır (Örnek 8). Hem hızlı arama hem hızlı ekleme için dengeli arama ağaçları (Veri Yapıları), yalnızca eşitlik araması için karma tabloları (H10) kullanılır. **Kural:** önce sorgu/güncelleme oranını ölçün, sonra veri yapısını seçin.

MÜHENDİS GÖZÜYLE · **Kanıt, test ve sınır değerleri**

- Bentley'nin ikili araması kitapta **kanıtlanmıştı**; hata, kanıtın matematiksel tamsayılar varsaydığı, programın ise 32 bitlik `int` kullandığı yerdeydi. Kanıtın modeli ile makinenin modeli aynı olmalıdır (Bloch, 2006).
- Sınır değerlerinde test edin: n = 0, 1, 2, ikinin kuvvetleri ve bir eksikleri, ilk ve son eleman, en küçükten küçük ve en büyükten büyük x, türün sınırına yakın indisler (Örnek 6: n = 2^{30} ve 2^{30} + 1).
- Kâhinle rastgele karşılaştırmalı test (differential testing) tek tek seçilmiş testlerin kaçırdığını yakalar: Örnek 7'de 200 rastgele dizi ve 10 000 sorgu, Örnek 9'da m = 1 … 100'ün bütün döndürmeleri.

10. Mini-OBS 2 v2.3: Numarayla İkili Arama ve Soyad Öneki

Bu hafta: Mini-OBS'nin kayıt dizisini numaraya göre sıralı tutuyor, numarayla bulmayı ikili aramaya çeviriyor, soyad önekiyle arama ekliyor ve doğrusal ile ikili aramayı 10^5 kayıtta karşılaştırmalı ölçüyoruz. Yükleyicinin mükerrer denetimi de ikili aramaya geçiyor; bunun karşılaştırmayı neden azalttığını ama süreyi neden azaltmadığını ölçerek göstereceğiz.

Proje bu haftanın dört fikrini kullanır: yarı açık ikili arama (Örnek 4), "var mı?" ve "nereye eklenir?" için lower_bound (Örnek 7, 8), önek denetimli soyad araması (Örnek 12) ve ölçüm düzeni (Bölüm 6). Proje belgesindeki referans ölçümlere göre 10^5 kayıtta numarayla arama doğrusal aramada 190,5 µs, ikili aramada 261 ns sürer (730 kat; 10^6 kayıtta 9 041 kat). Mükerrer denetimi ikili aramaya geçince karşılaştırma çok azalır ama süre azalmaz: her sıralı ekleme kayıtları kaydırır (Örnek 8'in $n^2/4$'ü).

Gereksinimler

- `arama.h/.c`: `orta_nokta`, `sinif_sirali_mi`, `ikili_ara_no_say` (yarı açık [lo, hi), erken çıkışlı), `ikili_ara_no_kapali_say` (kapalı [lo, hi]), `no_alt_sinir_say` (lower_bound), `sinif_sirali_ekle_say` (lower_bound + memmove; mükerrerde −1), soyad dizini (`soyad_dizini_kur`: upper_bound ile kararlı ekleme) ve `soyad_onek_ara` (lower_bound + önek denetimi). Her fonksiyonun başında ön/son koşul ve döngü değişmezi yorumu.
- Değişmez: `mini_obs`'un kayıt dizisi her an numaraya göre **kesin artan**dır. `sinif_yukle` sıralı kipte yükler; menüden ekleme sıralı konuma yapar; numarayla bulan her menü işlemi ikili arama kullanır.
- Orta nokta yalnızca `lo + (hi − lo) / 2` ile hesaplanır.
- Mükerrer numara satırı v1.0'daki **aynı satır numarasıyla** bildirilir.
- Menü 3: `*` ile biten metin soyad önekidir; sorgu `ad_normallestir` ile Türkçe büyük harfe çevrilir.
- `ara` programı: `--no N` (`--izle`, `--dogrusal`), `--soyad ÖNEK`, `--goster K`; çıkış kodları 0/1/2.
- `olcer`: en az `dogrusal`, `ikili`, `yukleme` ve `yukleme_v10`; `--sira artan|azalan` ile en iyi/en kötü durum; CSV'de karşılaştırma ve taşıma sayaçları.
- `test_obs.c`: elle izlenmiş inceleme sayıları; `test_kahin.c`: doğrusal kâhin, qsort kâhini, ters çevrim kâhini; `--hata-goster ikili`.
- Ölçüm raporu: arama başına süre ($n = 10 \ldots 10^6$, en az 10^5 dahil) ve yükleme süresi ($n = 10^3 \ldots \geq 6{,}4 \cdot 10^4$) tabloları, log–log grafikler, sayaçların kuramla karşılaştırılması ve yorum.

İpuçları

- İkili aramayı yazmadan önce **değişmezi** yazın ve her daraltmanın onu koruduğunu kâğıt üstünde gösterin. Yarı açık aralıkta `hi = orta`, kapalı aralıkta `hi = orta − 1`; karıştırmak ya kayıt kaçırır ya sonsuz döngü üretir.
- `lower_bound` hem “var mı?” hem “nereye eklenir?” sorusunu yanıtlar: `k < n && kayit[k].no == no` ise var; değilse ekleme konumu k'dir.
- Kaydırmayı döngüyle değil `memmove` ile yapın (örtüşen bölgede `memcpy` tanımsız davranıştır). Eklemeden önce kapasiteyi büyütmeyi unutmayın.
- Sıralı ekleme yaparken taşıma sayısını da sayın: karışık sırada $\approx n^2/4$ olduğunu görünce süre ölçümü sizi şaşırtmayacak.
- Önek aramasında karşılaştırıcı ile önek denetimi **aynı sırayı** kullanmalıdır (v2.3: bayt sırası, `strcmp`/`strncmp`).
- Sorguyu ve kayıtları aynı kuralla normalleştirin; “yıl” ile “YIL” aynı önek olmalıdır.
- Ölçüm verisini kurarken sıralı diziyi “hile” ile (proje belgesi, bölüm 6.8) kurmak ölçüm düzeneğidir; algoritma olarak kendi sıralı eklemenizi kullanın.

Kabul ölçütleri

1. `make` ve `make test` sıfır uyarıyla derlenir; v1.0'ın 99 doğrulaması ve önceki sürümlerin testleri dahil bütün birim testleri ve `test_kahin` (300 deneme) geçer.
2. `./ara ornek_veri/ogrenciler.csv --no 2301010050 --izle`: ortalar sırasıyla 4, 6, 5; kayıt 3 incelemede bulunur (sıra 6, EMRE ÇINAR); `--no 2301010044`: “bulunamadı (ekleme konumu: sıra 1)”.

3. `./ara ornek_veri/ogrenciler.csv --soyad yıl` → 2 kayıt (2301010045 AYŞE NUR YILMAZ, 2301010048 CAN YILMAZ), bu sırayla; `--soyad ç` → ZEYNEP ÇELİK ve EMRE ÇINAR; `--soyad i` → 0 kayıt ("İ" ile başlayan soyad yok).
4. 10^5 kayıt (`veri_uret --n 100000 --tohum 42`): `./ara u.csv --no 2400050000` en çok 17 incelemede bulur; `--dogrusal` ile doğrusal arama 50 000 karşılaştırma yapar.
5. Sırasız bir CSV `mini_obs` ile açılıp kaydedilince dosya numara sırasında yazılır; menü 2 listesi numara sırasındadır; menü 1 ile en küçük numarayı eklemek kaydı sıra 1'e koyar.
6. Mükerrer numaralı dosya v1.0 ile **aynı satır numarasıyla** reddedilir (ör. `testler/yinelenen_k3.csv` → "dosya biçimi hatalı (satır 12)" hem v1.0 hem v2.3).
7. Sayaçlar: `./olcer yukleme --sira azalan --n 1000` → taşıma 499 500 ve `kuram: UYUMLU`; `--sira artan` → taşıma 0, `kuram: UYUMLU`; `./olcer yukleme_v10 --n 1000` → karşılaştırma 499 500.
8. Kendi makinenizde 10^5 kayıtta ikili arama doğrusal aramadan en az 100 kat hızlıdır (bu makinede 730 kat).
9. Rapor, 10^5 kayıtlık karışık dosyada v2.3 yükleyicisinin karşılaştırma ve taşıma sayılarını ve süresini v1.0 ile karşılaştırır ve "neden hızlanmadı?" sorusunu taşınan bayt üzerinden yanıtlar.
10. `./test_kahin --hata-goster ikili` uyuşmazlığı ve küçültülmüş karşı örneği basar, 1 ile çıkar; `make asan` temizdir.

Tablo 4.13
Mini-OBS 2 v2.3 test senaryoları (proje belgesinden)

#	Girdi	Beklenen çıktı / davranış
1	`ara ornek_veri/ogrenciler.csv --no 2301010050 --izle`	3 adım (orta 4, 6, 5), sıra 6
2	aynı dosya, `--no 2301010099`	bulunamadı, ekleme konumu sıra 9; 3 inceleme
3	aynı dosya, `--no 2301010044`	bulunamadı, ekleme konumu sıra 1; 4 inceleme
4	aynı dosya, `--soyad yıl`	2 kayıt: ...045, ...048 (numara sırasında)
5	aynı dosya, `--soyad ''`	8 kayıt, bayt sırası: DOĞAN, KAYA, YILMAZ, YILMAZ, ÇELİK, ÇINAR, ÖZTÜRK, ŞAHİN
6	`mini_obs`, menü 3: `x*`	"Soyadı X ile başlayan kayıt yok."
7	`mini_obs`, menü 1: numara 2301010044	kayıt sıra 1'e eklenir; menü 2'de ilk satır
8	sırasız 3 kayıtlı CSV (30, 10, 20), menü 9	dosya 10, 20, 30 sırasıyla yazılır
9	`testler/bozuk.csv`	"dosya biçimi hatalı (satır 3)", çıkış 1
10	`olcer yukleme --sira azalan --n 1000,2000`	taşıma 499 500 ve 1 999 000; kuram UYUMLU
11	`olcer ikili --n 100000`	arama başına ortalama inceleme ≤ 17
12	`test_kahin --hata-goster ikili`	KÂHİN UYUŞMAZLIĞI; küçültülmüş örnek [1, 4], numara 1; çıkış 1

Öğrenciye verilenler (`arama.h`'deki bildirimler, sözleşmeler ve değişmezler; `olcer`'in ölçüm düzeneği), yazılacak modüller ve üç meydan okuma görevi Mini-OBS 2 v2.3 proje belgesindedir.

11. Alıştırmalar ve Atölye

Alıştırmalar yedi türdedir (A izleme/çıktı tahmini, B karmaşıklık, C değişmez yazma, D hata bulma, E Parsons, F kod yazma, G diller arası okuma). Kodlar `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 4.1 ★☆☆ *Çıktı tahmini*

Program incelediği her mid'i yazdırır. Üç satırın çıktısını çalıştırmadan yazın. Hangi aramada yoklama sayısı $\lfloor \log_2 10 \rfloor + 1 = 4$'e ulaşır?

alistirmalar/H04_A01_cikti_tahmini.c · satır 10–34

```c
static int ara(const int a[], int n, int x)
{
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        printf("%d ", mid);
        if (a[mid] == x) {
            return mid;
        } else if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return -1;
}

int main(void)
{
    const int a[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    printf("-> %d\n", ara(a, 10, 23));
    printf("-> %d\n", ara(a, 10, 7));
    printf("-> %d\n", ara(a, 10, 100));
    return 0;
}
```

Alıştırma 4.2 ★★☆ *İzleme*

Örnek 4'ün yarı açık aramasıyla (`c/H04_04_yari_acik_aralik.c`, 16 elemanlı izleme dizisi) x = 95 ve x = 2 için lo, hi, mid ve a[mid] tablosunu çıkarın. Her adımda üç bölgenin (< x, aday, > x) boyutlarını yazın ve değişmezin korunduğunu gösterin. Yoklama sayılarınızı programın “Yoklama:” satırıyla karşılaştırın.

11.2 Karmaşıklık (B)

Alıştırma 4.3 ★★☆ *Karmaşıklık*

Bu fonksiyon sıralı olmayan bir dizide “ikiye böl, iki yarıda da ara” yapar. Dizide olmayan bir x için çağrı sayısı C(n) bir yineleme bağıntısına uyar: yazın (n ikinin kuvveti varsayın) ve yerine koyma yöntemiyle

çözün. İkili aramanın T(n) = T(n/2) + c bağıntısından farkı nedir? Bu yöntem doğrusal aramadan daha mı hızlıdır? Sonra programı çalıştırıp sayaçla karşılaştırın.

alistirmalar/H04_A05_iki_yari.c · satır 14–26

```c
/* x, a[lo..hi-1] içinde var mı? Sıralama varsayımı YOK. */
static int iki_yari_ara(const int a[], size_t lo, size_t hi, int x)
{
    cagri++;
    if (hi - lo == 0) {
        return 0;
    }
    if (hi - lo == 1) {
        return a[lo] == x;
    }
    size_t mid = lo + (hi - lo) / 2;
    return iki_yari_ara(a, lo, mid, x) || iki_yari_ara(a, mid, hi, x);
}
```

Alıştırma 4.4 ★★☆ *Karmaşıklık*

Cevap üzerinde ikili arama ile bir kapasite problemi çözülecek (Kavram 7): yanıt aralığı [1, 10^9], her yüklem değerlendirmesi n = 10^5 elemanlı diziyi bir kez tarıyor. (a) İkili aramayla kaç yüklem değerlendirmesi ve toplam kaç eleman incelemesi gerekir? (b) Adayları 1'den başlayıp birer artırarak denemek en kötü durumda kaç inceleme ister? (c) Bölüm 6'da bu makinede ölçülen doğrusal tarama hızıyla (eleman başına ≈ 0,64 ns) iki yöntemin süresini kestirin.

11.3 Değişmez yazma (C)

Alıştırma 4.5 ★★☆ *Değişmez yaz*

Örnek 9'daki `en_kucuk_konum` (`c/H04_09_dondurulmus_dizi.c · satır 47–60`) döndürülmüş, farklı elemanlı bir dizide en küçük elemanın konumunu bulur. Değişmez "en küçük eleman a[lo..hi] içindedir" için başlatma, koruma ve sonlanmayı gösterin. Neden `hi = mid` yazılır, `hi = mid - 1` yazılmaz? Döngü neden `lo < hi` ile biter ve dönüş değeri neden doğrudur?

Alıştırma 4.6 ★★☆ *Değişmez yaz*

Merdiven aramasını sol alt köşeden başlatın: sözde kodda hangi durumda satır, hangi durumda sütun elenir? Değişmezi (x varsa hangi alt matristedir?) yazıp korunduğunu gösterin; en çok kaç adım sürer?

11.4 Hata bulma (D)

Alıştırma 4.7 ★★☆ *Hata bul*

Bu tamsayı karekök fonksiyonu bazı n'lerde bir eksik sonuç veriyor. Çalıştırmadan yanlış n'leri bulun; hatanın nedeni yüklemdir: Bölüm 3.4'teki "ilk D" ile karşılaştırıp tek bir karakteri silerek düzeltin.

alistirmalar/H04_A02_hata_bul_karekok.c · satır 10–23

```c
/* "İlk D" aranıyor: P(r) doğru olan en küçük r; yanıt r - 1 */
static long karekok(long n)
{
```

```
    long lo = 1, hi = n + 1;
    while (lo < hi) {
        long mid = lo + (hi - lo) / 2;
        if (mid * mid >= n) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo - 1;
}
```

Alıştırma 4.8 ★☆☆ *Hata bul*

Bir öğrenci kapalı aralıklı ikili aramada `lo = 0, hi = n - 1` ile başlayıp döngü koşulunu `while (lo < hi)` yazmış, güncellemeler `lo = mid + 1` ve `hi = mid - 1`. Hangi girdilerde yanlış sonuç alınır? En küçük karşı örneği (n ve x) verin. Atölye 1'in test paketi bu hatayı hangi kategoride yakalar?

11.5 Parsons (E)

Alıştırma 4.9 ★★☆ *Parsons*

Aşağıdaki satırlar upper_bound'un (ilk a[i] > x konumu) karıştırılmış satırlarıdır (dosya tamamlanacak; bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın, değişmezi yorum olarak ekleyin. Sonra Örnek 7'nin `ust_sinir` fonksiyonuyla karşılaştırın.

alistirmalar/H04_A03_parsons_ust_sinir.c · satır 6–18

```
            hi = mid;
    return lo;
    while (lo < hi) {
static size_t ust_sinir(const int a[], size_t n, int x)
        } else {
    size_t lo = 0, hi = n;
        size_t mid = lo + (hi - lo) / 2;
        }
        if (a[mid] <= x) {
    }
            lo = mid + 1;
{
}
```

11.6 Kod yazma (F)

Alıştırma 4.10 ★☆☆ *Kod yaz*

`long son_konum(const int a[], size_t n, int x)` fonksiyonunu upper_bound kullanarak yazın: x'in son konumu, yoksa −1. Örnek 7'nin dizisinde x = 0 … 13 için sonuçları Örnek 7'nin “son” sütunuyla karşılaştırın. C++ (`std::upper_bound` yalnızca kâhin olarak) ve Python (`bisect_right` yalnızca kâhin olarak) karşılıklarını da yazın.

Alıştırma 4.11 ★★☆ *Kod yaz*

Satır ve sütunları sıralı m × n matriste **kısa kenar boyunca** ikili arama yapan bir fonksiyon yazın: m ≤ n ise her satırda, m > n ise her sütunda ikili arama. Örnek 14'ün beş boyutunda (5 × 6, 10 × 10, 100 × 100, 4 × 1000, 1000 × 4) en çok adım sayısını ölçüp merdiven aramasıyla karşılaştıran bir tablo üretin. Hangi biçimde hangisi üstündür? Bir karar kuralı önerin.

Alıştırma 4.12 ★★★ *Kod yaz*

En büyük yükün en küçüğü. n kitabın sayfa sayıları raf sırasıyla bir dizide veriliyor; kitaplar bu sırayla, ardışık gruplar hâlinde k öğrenciye dağıtılacak. En çok sayfa okuyan öğrencinin sayfa sayısını en küçük yapan dağıtımda bu değer nedir? Cevap üzerinde ikili arama ile çözün (Kavram 7'nin kalıbı): yanıt aralığını ve monoton yüklemi açıkça yazın. n ≤ 12 için bütün bölmeleri deneyen bir kaba kuvvet kâhini yazıp 1000 rastgele girdide (ortak LCG) karşılaştırın; yüklem çağrısı sayısını raporlayın.

11.7 Diller arası okuma (G)

Alıştırma 4.13 ★★☆ *Diller arası okuma*

İki program iki orta nokta ifadesini üç (lo, hi) çifti için yazdırır. C programı ve altındaki Python programı ne yazdırır? Hangi satırlarda iki ifade C'de farklı, Python'da aynı sonucu verir? Nedenini H1'deki "sıfıra doğru kesme" ile açıklayın. İkili aramada lo ve hi negatif olmadığı sürece bu fark neden önemsizdir?

alistirmalar/H04_A04_orta_nokta.c · satır 13–18

```c
    const int lo[] = {0, -5, -7}, hi[] = {9, 0, -2};
    for (int i = 0; i < 3; i++) {
        int m1 = (lo[i] + hi[i]) / 2;
        int m2 = lo[i] + (hi[i] - lo[i]) / 2;
        printf("lo = %d, hi = %d: %d %d\n", lo[i], hi[i], m1, m2);
    }
```

Python **alistirmalar/H04_A04_orta_nokta.py · satır 10–14**

```python
    lo, hi = [0, -5, -7], [9, 0, -2]
    for i in range(3):
        m1 = (lo[i] + hi[i]) // 2
        m2 = lo[i] + (hi[i] - lo[i]) // 2
        print(f"lo = {lo[i]}, hi = {hi[i]}: {m1} {m2}")
```

Alıştırma 4.14 ★★☆ *Diller arası okuma*

Örnek 9'un Python sürümünde döndürülmüş dizinin sıralı yarısı `if a[lo] <= x < a[mid]:` ile sınanır. Bu satır C'ye harfi harfine `if (a[lo] <= x < a[mid])` diye çevrilirse derlenir mi, derleyici `-Wall` ile ne söyler? a = [40, 50, 60], lo = 0, mid = 2, x = 70 için C ifadesinin değeri nedir, Python'unki nedir? Doğru C yazımı nedir?

11.8 Atölye 1 · "Doğru ikili aramayı yaz" meydan okuması (bireysel, ≈ 30 dk)

Bentley'nin görevini deneyelim (Bölüm 1). Kurallar: **20 dakika**, internet ve ders notu kapalı, derleyici açık. x'in konumunu ya da −1'i döndüren ikili aramayı

`kodlar/alistirmalar/H04_A06_test_paketi.c`'deki `ogrenci_ara`'nın yerine yazın (Python: `H04_A06_test_paketi.py`). Diziye `a[i]` ile değil `eleman(i)` ile erişin ve her adımda `eleman(mid)`'i **bir kez** çağırın. Süre dolunca paketi çalıştırın: n = 0 … 64 ve x = −1 … 2n + 1 için 4355 sorgu yapar ve dört hata kategorisini sayar:

C alistirmalar/H04_A06_test_paketi.c · satır 25–37

```c
/* Diziye TEK erişim yolu: a[i] yerine eleman(i) yazın */
static int eleman(long i)
{
    yoklama++;
    if (yoklama > YOKLAMA_SINIRI) {
        longjmp(kacis, 1);      /* sonsuz döngü: aramadan çık */
    }
    if (i < 0 || (size_t)i >= dizi_n) {
        disari++;               /* gerçek bir dizide tanımsız davranış */
        return 0;
    }
    return dizi[i];
}
```

Tablo 4.14
Atölye 1 test paketinin hata kategorileri

Kategori	Nasıl yakalanır	Tipik neden
yanlış sonuç	dönüş değeri beklenen konumla (a[i] = 2i + 1) karşılaştırılır	sınır eleman kaçar (HA2 b), sırasız ön koşul
dizi dışı okuma	`eleman(i)` i < 0 ya da i ≥ n'yi sayar (gerçek dizide tanımsız davranış)	`hi = n` ile `<=` (HA2 a), `size_t` sarması (HA9)
sonsuz döngü	200 yoklamayı aşan arama `longjmp` ile kesilir	`lo = mid` (HA1), aralık daralmıyor
çok yoklama	yoklama sayısı ⌊log₂ n⌋ + 1 ile karşılaştırılır	ikili değil doğrusal arama; mid'i iki kez okumak

ÇIKTI · YER TUTUCU (DOĞRUSAL ARAMA) İLE

```
$ ./H04_A06_test_paketi
n = 0..64, x = -1..2n+1: 4355 sorgu
  yanlış sonuç                    : 0
  dizi dışı okuma yapan arama     : 0
  200 yoklamayı aşan (sonsuz döngü)  : 0
  ⌊log2 n⌋ + 1 yoklamayı aşan     : 4015
ilk sorun: n = 3, x = -1 (çok yoklama)
sonuç: KALDI
```

Yer tutucu doğru ama yavaştır (yalnızca “çok yoklama”). Sınıfın sonuçlarını tahtada toplayın: kaç kişi ilk denemede GEÇTİ, en sık hangi kategori görüldü? Tartışma: (1) Hangi sözleşmeyi seçtiniz, dört öge tutarlı mı (Tablo 4.1)? (2) Testten önce kaç sınır durumunu düşündünüz? (3) $n = 2^{30}$ olsaydı orta noktanız ne yapardı?

11.9 Atölye 2 · Sayı tahmin oyunu ve soru sayısı istatistiği (eşli, ≈ 20 dk)

`kodlar/alistirmalar/H04_A07_sayi_tahmin.c` iki kipte çalışır: **bilgisayar** kipinde program 1..N'de bir sayı tutar (ortak LCG, tohum komut satırından) ve siz tahmin edersiniz; **ogrenci** kipinde siz tutarsınız, program ikili aramayla sorar, siz b (büyük), k (küçük) ya da d (doğru) dersiniz:

ÖRNEK OTURUM · ÖĞRENCİ 35'İ TUTAR

```
$ ./H04_A07_sayi_tahmin ogrenci
1..100 arasında bir sayı tutun. Yanıtlar: b / k / d
  soru 1: 50 mi? k
  soru 2: 25 mi? b
  soru 3: 37 mi? k
  soru 4: 31 mi? b
  soru 5: 34 mi? b
  soru 6: 35 mi? d
buldum: 35 (6 soru; en çok 7)
```

- **Tur 1 (bilgisayar tutar).** N = 100 ile on oyun oynayın (`./H04_A07_sayi_tahmin bilgisayar 100 T`, T = 1 … 10), soru sayılarını yazın; ortalamanızı ve en çoğunuzu Örnek 10'un dağılımıyla (1, 2, 4, 8, 16, 32, 37 oyun; ortalama 5,80; en çok 7) karşılaştırın.
- **Tur 2 (öğrenci tutar).** Eşiniz tutsun, program sorsun. Bir kez "yanlış" yanıt verilirse ne olur? (Aralık boşalınca program "yanıtlar çelişkili" der: değişmez bozulmuştur.)
- **Tartışma.** N = 1000 ve 10^6 için en çok soruyu önceden kestirip sınayın. Ortalama neden en çoğun yaklaşık bir eksiğidir? Yalnızca "x ≤ m mi?" (evet/hayır) sorulsaydı en çok kaç soru gerekirdi?

11.10 Atölye 3 · Varyant avı: lower_bound ile beş problem (eşli, ≈ 25 dk)

`kodlar/alistirmalar/H04_A08_varyant_avi.c` beş problemin her biri için bir doğrusal kâhin, bir test düzeneği ve sizin dolduracağınız bir fonksiyon içerir. Şimdilik her fonksiyon kâhini çağırır: sonuçlar doğrudur ama yoklama sınırı aşılır. Her fonksiyonu lower_bound ile O(log n)'de yeniden yazın; 1–3 için yalnızca verilen `alt_sinir` çağrılarını kullanın, 4–5 için `alt_sinir`'in koşulu değiştirilmiş bir kopyasını yazın.

Tablo 4.15
Atölye 3'ün beş problemi

#	Fonksiyon	Soru	Yoklama sınırı (n = 1000)
1	`adet(a, n, x)`	x kaç kez geçer?	$2(\lfloor \log_2 n \rfloor + 1)$
2	`aralik_say(a, n, L, R)`	L ≤ a[i] ≤ R olan kaç eleman var?	$2(\lfloor \log_2 n \rfloor + 1)$
3	`en_yakin(a, n, x)`	x'e en yakın eleman (eşit uzaklıkta küçük olan)	$\lfloor \log_2 n \rfloor + 3$
4	`eksik_sayi(a, n)`	1..n + 1'den biri eksik sıralı dizide eksik sayı	$\lfloor \log_2 n \rfloor + 1$
5	`ilk_kotu(k, kotu)`	1..k sürümde ilk kötü sürüm (git bisect)	$\lfloor \log_2 k \rfloor + 1$ ($k = 10^6$)

Yer tutucularla program beş problemin hepsinde doğru sonuç verir ama yoklama sınırını aşar; örneğin `adet` 224/224 testte doğrudur, ama en çok 1000 yoklama yapar (sınır 20): beşi de KALDI.

İpuçları: (1) ve (2) iki sınır farkıdır; tamsayılarda "≤ R", "< R + 1" ile aynıdır. (3) lower_bound'un gösterdiği eleman ve bir öncekisi tek adaylardır; uçlara dikkat. (4) a[i] > i + 1 yüklemi eksik sayıdan sonra

hep doğrudur. (5) `kotu` bir fonksiyon işaretçisidir. Bitirenler: eşitliği bilerek yanlış tarafa koyup hangi testin bozulduğunu gözlesin.

12. Kendini Değerlendirme

Aşağıdakilerin her birini kitaba bakmadan yapabiliyorsanız işaretleyin; yapamadıklarınız için parantez içindeki bölüme dönün.

- ☐ Doğrusal aramayı nöbetçiyle de yazıp en iyi/kötü/ortalama karşılaştırmayı bulabiliyorum. (4.1)
- ☐ Karar ağacıyla neden $\lfloor \log_2 n \rfloor + 1$ sorunun hem yettiğini hem gerektiğini açıklayabiliyorum. (3.2)
- ☐ Kapalı ve yarı açık aralık sözleşmelerinin dört ögesini (başlangıç, koşul, iki güncelleme) tutarlı yazabiliyorum. (3.3, HA1, HA2)
- ☐ İkili aramada değişmezi (başlatma–koruma–sonlanma) ve sonlanmayı kanıtlayabiliyorum. (4.2)
- ☐ Orta nokta taşmasının nedenini ve C, C++, Python farkını açıklayabiliyorum. (1, 5.1, 8, HA3)
- ☐ lower_bound/upper_bound ile ilk/son konum, adet, ekleme konumu bulabiliyorum. (4.3, 5.2)
- ☐ Bir optimizasyon problemini monoton yükleme ve cevap üzerinde ikili aramaya çevirebiliyorum; ikiye bölmenin durma koşullarını yazabiliyorum. (3.4, 4.4, 4.5)
- ☐ Fonksiyon işaretçisi ve `void *` ile genel bir arama yazabiliyor, `bsearch`, `std::lower_bound` ve `bisect` karşılıklarını kullanabiliyorum. (5.4, 8)
- ☐ Ölçüm verisinden kesişim noktasını okuyup sınırlılıklarıyla birlikte yorumlayabiliyorum. (6)

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Bu haftanın konusu nerede derinleşiyor?**

- İkili arama ağaçları ve dengeli ağaçlar, Veri Yapıları dersinde sıralı dizinin dinamik karşılığı olarak ele alınır. Sıralı dizinin O(n) eklemesine (Örnek 8) verilen yanıt budur: arama, ekleme ve silme O(log n).
- H5'te araya ekleme sıralaması, Örnek 8'in sıralı eklemesinin kendisidir; ekleme konumunu ikili aramayla bulmak karşılaştırmayı azaltır, kaydırmayı azaltmaz. H10'da karma tabloları eşitlik aramasını ortalama O(1)'e indirir ama sıra bilgisini (lower_bound, aralık sorgusu) kaybeder.
- Cevap üzerinde ikili arama; optimizasyon problemlerinde ve yarışma programlamada sık kullanılır; H13'te en uzun artan alt dizinin hızlı sürümü lower_bound kullanır.
- git bisect: yazılım geçmişinde hatayı ikili aramayla bulma (Mühendis Gözüyle).

MERAK KUTUSU · **Fonksiyonu fonksiyona vermek ve üçe bölmenin neden kazandırmadığı**

İkiye bölmeye fonksiyon işaretçisi. Örnek 11'de denklem küresel bir seçiciyle değişiyordu. Kavram 6 yöntemi `typedef double (*fonk_t)(double);` türünde bir parametreyle yazar (`ikiye_bolme(fonk_t f, double a, double b, double tol, int *yineleme)`); tek fonksiyon her denklemi çözer. `kup` ve `kare` ayrı fonksiyonlardır, işaretçileri bir dizide durur; `g(3.0)` ile `(*g)(3.0)` aynı çağrıdır.

kavram/H04_K06_fonksiyon_isaretcisi.c · satır 47–55

```
47      const struct {
48          const char *ad;
49          fonk_t f;
50      } denklem[] = {{"x^3 - x - 2", kup}, {"x^2 - 2", kare}};
51      for (int i = 0; i < 2; i++) {
52          int k;
53          double r = ikiye_bolme(denklem[i].f, 1.0, 2.0, 1e-9, &k);
54          printf("%-12s kök ≈ %.9f  (%d yineleme)\n", denklem[i].ad, r, k);
55      }
```

> Programın ilk satırı: `x^3 - x - 2 kök ≈ 1.521379707 (30 yineleme)`. **Üçlü arama** (ternary search) aralığı iki noktayla üçe böler: bir adımda iki karşılaştırma yapar ve aralığı 1/3'e indirir. n eleman için $2 \cdot \log_3 n = (2 / \log_2 3) \cdot \log_2 n \approx 1{,}26 \cdot \log_2 n$ karşılaştırma eder: ikili aramadan **daha çok**. Karar ağacı alt sınırı (Bölüm 3.2) bu yüzden ikiye bölmeyi en iyi yapar. **İnterpolasyon** araması ise karşılaştırma modelinin dışına çıkar: değerlerin kendisinden aritmetik yaparak düzgün veride log log n mertebesine iner (Bölüm 5.6).

14. Özet, Kavram Haritası ve Terim Sözlüğü

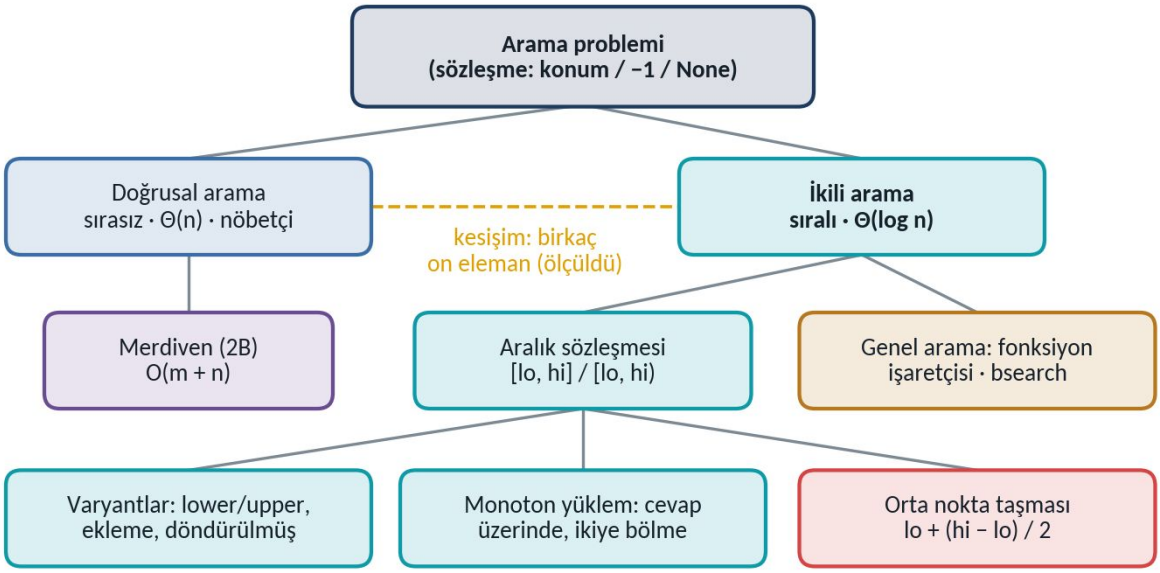
- Arama bir sözleşmedir: bulunamadı nasıl bildirilir (−1, `None`, istisna), tekrar eden değerlerde hangi konum, ön koşul ne?
- Doğrusal arama Θ(n) ve sırasız veride en iyisidir; nöbetçi karşılaştırma sayısını 2n + 1'den n + 2'ye indirir ama süre kazancı dile bağlıdır.
- İkili arama her yoklamada adayların yarısını eler: en çok $\lfloor \log_2 n \rfloor + 1$ yoklama (10^6 için 20) ve bu, karar ağacına göre her karşılaştırmalı arama için alt sınırdır.
- Doğruluk aralık sözleşmesinden gelir: kapalı [lo, hi] ya da yarı açık [lo, hi); dört öge uyumlu olmalı, aralık her adımda kesin daralmalıdır. Yarı açıkta döngü sonunda lo ekleme konumudur.
- Orta nokta `lo + (hi - lo) / 2`: C/C++'ta işaretli taşma tanımsız davranıştır; Python taşmaz.
- lower_bound / upper_bound eşitlikte durmaz: ilk/son konum, adet, ekleme konumu. Monoton yüklemde "ilk D" bakışı karekökü, kapasiteyi, ikiye bölmeyi ve git bisect'i aynı algoritmaya bağlar.
- Merdiven araması satırları ve sütunları sıralı matriste O(m + n)'dir; genel arama fonksiyon işaretçisi ve öge boyutuyla her türde çalışır.
- Bu makinede doğrusal arama C'de birkaç on elemana kadar ikiliden hızlıydı; Python'da düz doğrusal aramanın küçük-n üstünlüğü görülmedi. Kesişim, ölçüm koşuluna bağlı bir aralıktır.

Tablo 4.16
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
arama problemi; doğrusal arama; nöbetçi	search problem; linear search; sentinel	
ikili arama; yoklama; orta nokta	binary search; probe; midpoint	yoklama: a[mid]'e bakış
azalt ve fethet; karar ağacı	decrease and conquer; decision tree	
kapalı / yarı açık aralık; aralık sözleşmesi	closed / half-open interval; interval contract	
döngü değişmezi; ön koşul; son koşul	loop invariant; precondition; postcondition	Yol Haritası Bölüm 6.8
lower_bound/upper_bound; ekleme konumu	lower bound / upper bound; insertion point	"alt sınır" yalnızca Ω için
cevap üzerinde ikili arama; monoton yüklem	binary search on the answer; monotone predicate	
üstel arama; interpolasyon araması; üçlü arama	exponential (galloping) search; interpolation search; ternary search	kısa bakış
ikiye bölme yöntemi; merdiven araması	bisection method; staircase (saddleback) search	
fonksiyon işaretçisi; karşılaştırıcı; katı zayıf sıralama	function pointer; comparator; strict weak ordering	
taşma; tanımsız / uygulama tanımlı davranış	overflow; undefined / implementation-defined behavior	

Türkçe	İngilizce	Not
ön işleme; ödünleşim; kâhin	preprocessing; trade-off; test oracle	“takas” yalnızca swap
dal tahmini; önbellek	branch prediction; cache	Kaputun Altında

Şekil 4.17
Hafta 4 kavram haritası



15. Kaynaklar

Bentley, J. (2000). *Programming pearls* (2nd ed.). Addison-Wesley.

Bloch, J. (2006, June 2). *Extra, extra – read all about it: Nearly all binary searches and mergesorts are broken*. Google Research Blog. https://research.google/blog/extra-extra-read-all-about-it-nearly-all-binary-searches-and-mergesorts-are-broken/

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Free Software Foundation. (t.y.). *Integers implementation* (Using the GNU Compiler Collection). Erişim tarihi 13 Ekim 2026, https://gcc.gnu.org/onlinedocs/gcc/Integers-implementation.html

Git. (t.y.). *git-bisect: Use binary search to find the commit that introduced a bug*. Erişim tarihi 13 Ekim 2026, https://git-scm.com/docs/git-bisect

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2nd ed.). Addison-Wesley.

Python Software Foundation. (t.y.). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on dört basamağının C++ ve Python sürümlerinden çekirdek fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`). Fonksiyon ve değişken adları üç dilde aynıdır; C++ sürümlerinde yorumlar `//` ile yazılmıştır. Amiral örneklerin C++'ta farklılaşan satırları Bölüm 4'tedir; C++ çekirdek fonksiyonları C'ninkilerle satır satır aynı yapıdadır. Burada Python fonksiyonları, C++'ın öteki fonksiyonları ve her örneğin dil notu verilir.

Tablo 4.17
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H04_03_ikili_arama)

Dil	Derleme / çalıştırma	Ek dosya ya da not
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c -o X · ./X`	Örnek 3, 7, 8, 10, 14: + ortak_rastgele.c; Örnek 13: + zamanlayici.c
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Hepsi	`Örnek 4: X bozuk (assert durdurur); Örnek 13: X --olc`	süreler stderr'e yazılır; stdout üç dilde aynı

A.1 Örnek 1 · Doğrusal arama: ilk konum, bulunamazsa −1 / None; sıralı dizide erken çıkış

C++ cpp/H04_01_dogrusal_arama.cpp · satır 37–46

```cpp
static long sirali_dogrusal_ara(const std::vector<int>& a, int x)
{
    for (std::size_t i = 0; i < a.size(); i++) {
        karsilastirma++;                // bir eleman: bir sayım
        if (a[i] >= x) {
            return a[i] == x ? static_cast<long>(i) : -1;
        }
    }
    return -1;
}
```

Python python/H04_01_dogrusal_arama.py · satır 28–34

```python
def sirali_dogrusal_ara(a, x):
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1              # bir eleman: bir sayım
        if a[i] >= x:
            return i if a[i] == x else None
    return None
```

Farklılaşan satırlar. C ve C++'ta "bulunamadı" sözleşmesi bir nöbetçi değerdir (`-1`); Python'da `None` döndürülür ve `k is None` ile sınanır. C'de dizi ve boyutu ayrı parametredir (`const int a[], size_t`

n); C++'ta `const std::vector<int>&` ve `a.size()`, Python'da `len(a)` kullanılır. Kütüphane karşılıkları da farklı sözleşmeler taşır: `std::find` bulamazsa `end()` döndürür, Python'un `a.index(x)`'i `ValueError` fırlatır (Hata Avcısı 6). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 22, 25, 95; Python 18, 22, 43.) Üç dil kesiti: `dogrusal_ara`: Bölüm 4.1.

A.2 Örnek 2 · Nöbetçili doğrusal arama: sınır denetimi kalkar, karşılaştırma sayısı 2n + 1'den n + 2'ye iner

C++ **cpp/H04_02_nobetcili_arama.cpp · satır 37–50**

```cpp
static long nobetcili_ara(std::vector<int>& a, std::size_t n, int x)
{
    int eski = a[n];
    a[n] = x;               // nöbetçi
    std::size_t i = 0;
    eleman_kars++;
    while (a[i] != x) {
        i++;
        eleman_kars++;
    }
    a[n] = eski;            // geri yükle
    indis_kars++;           // tek sınır denetimi
    return i < n ? static_cast<long>(i) : -1;
}
```

Python **python/H04_02_nobetcili_arama.py · satır 29–40**

```python
def nobetcili_ara(a, n, x):
    global indis_kars, eleman_kars
    eski = a[n]
    a[n] = x                # nöbetçi
    i = 0
    eleman_kars += 1
    while a[i] != x:
        i += 1
        eleman_kars += 1
    a[n] = eski             # geri yükle
    indis_kars += 1         # tek sınır denetimi
    return i if i < n else None
```

Farklılaşan satırlar. Nöbetçi için dizinin sonunda bir boş yer gerekir: C'de kapasitesi n + 1 olan dizi, C++'ta n + 1 boyutlu `std::vector` (`push_back`, `assign`), Python'da sonuna `[0]` eklenmiş liste. Nöbetçi diziyi geçici olarak değiştirdiğinden parametre `const` olamaz. Python sürümü C ile aynı karşılaştırmaları saymak için `while` kullanır; `for i in range(n)` sınır denetimini yorumlayıcıya bırakırdı. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 72, 93; Python 59, 74.)

A.3 Örnek 3 · İkili arama — yinelemeli, kapalı aralık [lo, hi]; yoklama sayısı ≤ ⌊log₂ n⌋ + 1

Python **python/H04_03_ikili_arama.py · satır 16–28**

```python
def ikili_ara(a, x):
    global yoklama
    lo, hi = 0, len(a) - 1
```

```
    while lo <= hi:
        mid = (lo + hi) // 2    # taşma yok
        yoklama += 1
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1    # a[lo..mid] elendi
        else:
            hi = mid - 1    # a[mid..hi] elendi
    return -1                           # aralık boş: lo == hi + 1
```

Farklılaşan satırlar. Kapalı aralıkta `hi = n - 1` boş dizide −1 olur: C ve C++'ta `lo`, `hi`, `mid` işaretsiz `size_t` değil işaretli `long` tutulur (Hata Avcısı 9). Python'da `(lo + hi) // 2` güvenlidir, çünkü tamsayılar taşmaz; C/C++'ta taşmasız `lo + (hi - lo) / 2` yazılır (Örnek 6). Çoklu atama `lo, hi = 0, len(a) - 1` Python'a özgüdür. (Gösterilen fonksiyonların dışındaki fark satırları: Python 89.) Üç dil kesiti: C++ farkları: Bölüm 4.2.

A.4 Örnek 4 · İkili arama — yarı açık aralık [lo, hi) ve değişmez assert'leri; ekleme konumu

C++ **cpp/H04_04_yari_acik_aralik.cpp · satır 45–65**

```
static long ikili_ara_yari(const std::vector<int>& a, int x)
{
    std::size_t lo = 0, hi = a.size();  // başlangıç: tüm dizi [0, n)
    while (lo < hi) {                   // aralık boş değil
        assert(degismez_mi(a, lo, hi, x));
        std::size_t mid = lo + (hi - lo) / 2;   // lo <= mid < hi
        std::size_t eski = hi - lo;
        yoklama++;
        if (a[mid] == x) {
            return static_cast<long>(mid);
        } else if (a[mid] < x) {
            lo = mid + 1;               // a[lo..mid] < x
        } else {
            hi = mid;                   // a[mid..hi-1] > x
        }
        assert(hi - lo < eski);         // sonlanma: aralık daraldı
        (void)eski;                     // -DNDEBUG ile uyarı olmasın
    }
    assert(degismez_mi(a, lo, hi, x));
    return -1;                          // lo == hi: x buraya eklenir
}
```

Python **python/H04_04_yari_acik_aralik.py · satır 33–49**

```
def ikili_ara_yari(a, x):
    global yoklama
    lo, hi = 0, len(a)                  # başlangıç: tüm dizi [0, n)
    while lo < hi:                      # aralık boş değil
        assert degismez_mi(a, lo, hi, x), "değişmez bozuldu"
        mid = (lo + hi) // 2            # lo <= mid < hi
        eski = hi - lo
        yoklama += 1
```

```
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1                # a[lo..mid] < x
        else:
            hi = mid                    # a[mid..hi-1] > x
        assert hi - lo < eski, "aralık daralmadı"   # sonlanma
    assert degismez_mi(a, lo, hi, x), "değişmez bozuldu"
    return -1                           # lo == hi: x buraya eklenir
```

Farklılaşan satırlar. C'de `assert <assert.h>` makrosudur ve `-DNDEBUG` ile derlenince kaybolur; C++'ta aynı makro `<cassert>` ile gelir; Python'da `assert` bir deyimdir ve `python3 -O` ile kapanır. Başarısız assert C/C++'ta süreci `abort` ile durdurur (SIGABRT; kabukta çıkış 134), Python'da `AssertionError` fırlatır (çıkış 1): `bozuk` argümanıyla görülebilir. Değişmez denetimi O(n) maliyetlidir; yalnızca geliştirme sırasında açık tutulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 11.)

A.5 Örnek 5 · İkili arama — özyinelemeli: T(n) = T(n/2) + c; çağrı ve derinlik sayaçları

C++ **cpp/H04_05_ozyinelemeli_ikili.cpp · satır 19–37**

```
static long ikili_ara_oz(const std::vector<int>& a, std::size_t lo,
                         std::size_t hi, int x, int derinlik)
{
    cagri++;
    if (derinlik > en_derin) {
        en_derin = derinlik;
    }
    if (lo >= hi) {                     // temel durum: boş aralık
        return -1;
    }
    std::size_t mid = lo + (hi - lo) / 2;
    if (a[mid] == x) {                  // temel durum: bulundu
        return static_cast<long>(mid);
    }
    if (a[mid] < x) {                   // sağ yarı: [mid + 1, hi)
        return ikili_ara_oz(a, mid + 1, hi, x, derinlik + 1);
    }
    return ikili_ara_oz(a, lo, mid, x, derinlik + 1);   // [lo, mid)
}
```

Python **python/H04_05_ozyinelemeli_ikili.py · satır 14–26**

```
def ikili_ara_oz(a, lo, hi, x, derinlik):
    global cagri, en_derin
    cagri += 1
    if derinlik > en_derin:
        en_derin = derinlik
    if lo >= hi:                        # temel durum: boş aralık
        return -1
    mid = (lo + hi) // 2
    if a[mid] == x:                     # temel durum: bulundu
        return mid
```

```
    if a[mid] < x:                      # sağ yarı: [mid + 1, hi)
        return ikili_ara_oz(a, mid + 1, hi, x, derinlik + 1)
    return ikili_ara_oz(a, lo, mid, x, derinlik + 1)    # [lo, mid)
```

Farklılaşan satırlar. Özyinelemeli çağrılar kuyruk konumundadır (çağrının sonucu doğrudan döndürülür); C/C++ derleyicileri eniyilemeyle bunu döngüye çevirebilir, Python çevirmez. Derinlik en çok $\lfloor \log_2 n \rfloor + 2$ olduğundan ($n = 10^6$ için 21) Python'un özyineleme sınırı (1000) burada sorun olmaz. C++'ta dizi `const std::vector<int>&` ile kopyalanmadan aktarılır; Python'da girinti `"  " * (derinlik - 1)` dizgi tekrarıyla üretilir. (Gösterilen fonksiyonların dışındaki fark satırları: Python 31.)

A.6 Örnek 6 · Orta nokta taşması: (lo + hi) / 2 ile int32 sınırında hata ve güvenli lo + (hi − lo) / 2

C++ cpp/H04_06_orta_nokta_tasmasi.cpp · satır 29–54

```cpp
static std::int32_t sarmali_orta(std::int32_t lo,
                                 std::int32_t hi)
{
    auto ulo = static_cast<std::uint32_t>(lo);
    auto uhi = static_cast<std::uint32_t>(hi);
    std::uint32_t toplam = ulo + uhi;   // mod 2^32
    return static_cast<std::int32_t>(toplam) / 2;
}

// Güvenli: 0 <= lo <= hi iken hi - lo taşmaz, sonuç [lo, hi] içinde
static std::int32_t guvenli_orta(std::int32_t lo,
                                 std::int32_t hi)
{
    return lo + (hi - lo) / 2;
}

// Bloch'un (2006) ikinci önerisi: işaretsiz toplam, 1 bit sağa kaydırma
// (Java'da >>>). Toplam < 2^32 olduğundan sonuç int32_t'ye sığar.
static std::int32_t kaydirmali_orta(std::int32_t lo,
                                    std::int32_t hi)
{
    auto ulo = static_cast<std::uint32_t>(lo);
    auto uhi = static_cast<std::uint32_t>(hi);
    std::uint32_t toplam = ulo + uhi;
    return static_cast<std::int32_t>(toplam >> 1);
}
```

Python python/H04_06_orta_nokta_tasmasi.py · satır 20–46

```python
def sifira_dogru_bol(a, b):
    q = abs(a) // abs(b)
    return q if (a >= 0) == (b > 0) else -q


# (lo + hi) / 2'nin 32 bitlik ikiye tümleyen makinedeki sonucunun
# TAKLİDİ: toplam 2^32 modunda alınır, işaretli aralığa kaydırılır.
def sarmali_orta(lo, hi):
    toplam = (lo + hi) & 0xFFFFFFFF  # uint32_t toplamı
    if toplam > INT32_MAX:
```

```
        toplam -= 2**32                 # int32_t'ye
    return sifira_dogru_bol(toplam, 2)


# Güvenli: 0 <= lo <= hi iken sonuç [lo, hi] içinde (her dilde)
def guvenli_orta(lo, hi):
    return lo + (hi - lo) // 2


# Bloch'un (2006) ikinci önerisi: işaretsiz toplam, 1 bit sağa kaydırma
def kaydirmali_orta(lo, hi):
    return ((lo + hi) & 0xFFFFFFFF) >> 1


# Python'un kendi tamsayısıyla: taşma yok, sonuç her zaman doğru
def python_orta(lo, hi):
    return (lo + hi) // 2
```

Δ — dile özgü çıktı. C ve C++'ta `int32_t` toplamının taşması tanımsız davranıştır; örnek bu yüzden toplamı işaretsiz `uint32_t` ile (2^{32} modunda, tanımlı) hesaplar ve sonucu `int32_t`'ye dönüştürür: bu dönüşüm C17'de ve C++17'de uygulama tanımlıdır (GCC 2^{32} modunda indirger; C++20'den beri tanımlı). Python tamsayıları keyfi duyarlıklıdır: `(lo + hi) // 2` her zaman doğrudur; 32 bitlik sarma ancak `& 0xFFFFFFFF` ile taklit edilir, C'nin sıfıra doğru kesen bölmesi için de `//` yerine `sifira_dogru_bol` gerekir. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 119.)

C++ çıktısı C çıktısıyla bayt bayt aynıdır; ilk ve son bölümü Bölüm 5.1'dedir, orta bölümü:

ÇIKTI · C VE C++ (ORTA BÖLÜM)

```
$ ./H04_06_orta_nokta_tasmasi
Sanal dizide (n = 2000000000) x = n - 1 aranıyor
sarmalı orta nokta (32 bit int ile tipik sonuç):
  adım 1: lo = 0, hi = 1999999999, mid = 999999999
  adım 2: lo = 1000000000, hi = 1999999999, mid = -647483648
  mid aralık dışında (negatif indis): a[mid] okunsaydı
  dizi dışı erişim olurdu; durduruldu
güvenli orta nokta:
  31 adımda bulundu: konum 1999999999
```

Python çıktısı (tamamı):

ÇIKTI · PYTHON

```
$ python3 H04_06_orta_nokta_tasmasi.py
INT32_MAX = 2147483647
lo = 1500000000, hi = 2000000000
gerçek lo + hi               = 3500000000
(lo + hi) // 2 (Python)      = 1750000000   <- taşma yok
(lo + hi) / 2, 32 bit taklit = -397483648
lo + (hi - lo) // 2          = 1750000000
(lo + hi) >> 1 (32 bit)      = 1750000000

Sanal dizide (n = 2000000000) x = n - 1 aranıyor
Python (lo + hi) // 2:
  31 adımda bulundu: konum 1999999999
```

```
32 bit sarma taklidi (C/C++'taki tipik sonuç):
  adım 1: lo = 0, hi = 1999999999, mid = 999999999
  adım 2: lo = 1000000000, hi = 1999999999, mid = -647483648
  mid aralık dışında (negatif indis): a[mid] okunsaydı
  dizi dışı erişim olurdu; durduruldu
güvenli orta nokta:
  31 adımda bulundu: konum 1999999999

Hata hangi n'de başlar? (son eleman aranırken; C/C++ int32)
  n = 1073741824: en büyük lo + hi = 2147483646  <= INT32_MAX: taşmaz
  n = 1073741825: en büyük lo + hi = 2147483648  > INT32_MAX: TAŞAR
```

A.7 Örnek 7 · lower_bound ve upper_bound; ilk/son konum ve bir değerin kaç kez geçtiği

C++ **cpp/H04_07_alt_ust_sinir.cpp · satır 38–51**

```cpp
static std::size_t ust_sinir(const std::vector<int>& a, int x)
{
    std::size_t lo = 0, hi = a.size();
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] <= x) {  // tek fark: <=
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}
```

Python **python/H04_07_alt_ust_sinir.py · satır 18–43**

```python
def alt_sinir(a, x):
    global yoklama
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        yoklama += 1
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo


# upper_bound: ilk a[i] > x konumu; hepsi <= x ise n.
# Değişmez: a[0..lo-1] <= x < a[hi..n-1]
def ust_sinir(a, x):
    global yoklama
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
```

```
        yoklama += 1
        if a[mid] <= x:     # tek fark: <=
            lo = mid + 1
        else:
            hi = mid
    return lo
```

Farklılaşan satırlar. C standart kütüphanesinde lower_bound yoktur (`bsearch` eşitlerden herhangi birini döndürür); C'deki kâhin doğrusal sayımdır. C++'ta `std::lower_bound` / `std::upper_bound` yineleyici döndürür, konum `it - a.begin()` ile bulunur; Python'da `bisect_left` / `bisect_right` doğrudan indis döndürür. İki fonksiyon arasındaki tek fark `a[mid] < x` ile `a[mid] <= x` karşılaştırmasıdır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 58, 60, 132; Python 9, 48.) Üç dil kesiti: C++ `alt_sinir` farkları: Bölüm 4.3.

A.8 Örnek 8 · Ekleme konumu ve sıralı diziye ekleme: arama O(log n), kaydırma O(n)

C++ **cpp/H04_08_ekleme_konumu.cpp · satır 21–50**

```cpp
static std::size_t ekleme_konumu(const std::vector<int>& a, int x)
{
    std::size_t lo = 0, hi = a.size();
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}

// Ön koşul: a sıralı. x'i yerine ekler ve konumunu döndürür.
// Sona bir yer açılır (push_back), a[k..n-1] bir sağa kayar:
// SONDAN başa doğru (üzerine yazmamak için).
static std::size_t sirali_ekle(std::vector<int>& a, int x)
{
    std::size_t k = ekleme_konumu(a, x);
    std::size_t n = a.size();
    a.push_back(x);                     // yeni boyut n + 1
    for (std::size_t i = n; i > k; i--) {
        a[i] = a[i - 1];
        kaydirma++;
    }
    a[k] = x;
    return k;
}
```

Python **python/H04_08_ekleme_konumu.py · satır 17–42**

```python
def ekleme_konumu(a, x):
    global yoklama
```

```
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        yoklama += 1
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo


# Ön koşul: a sıralı. x'i yerine ekler ve konumunu döndürür.
# Sona bir yer açılır (append), a[k..n-1] bir sağa kayar:
# SONDAN başa doğru (üzerine yazmamak için).
def sirali_ekle(a, x):
    global kaydirma
    k = ekleme_konumu(a, x)
    n = len(a)
    a.append(x)                          # yeni boyut n + 1
    for i in range(n, k, -1):
        a[i] = a[i - 1]
        kaydirma += 1
    a[k] = x
    return k
```

Farklılaşan satırlar. C'de dizinin kapasitesi sabittir (`KAPASITE`) ve yeni boyutu çağıran tutar; C++'ta `push_back`, Python'da `append` sona bir yer açar, kaydırma yine elle yapılır (`std::vector::insert` ve `list.insert` aynı kaydırmayı kendi içinde yapar). Python'da geriye sayan döngü `range(n, k, -1)` ile yazılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 107.)

A.9 Örnek 9 · Döndürülmüş sıralı dizide arama; döndürme noktası; tekrar eden elemanlarda O(n)

C++ **cpp/H04_09_dondurulmus_dizi.cpp · satır 20–44**

```
static long dondurulmus_ara(const std::vector<int>& a, int x)
{
    long lo = 0, hi = static_cast<long>(a.size()) - 1;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] == x) {
            return mid;
        }
        if (a[lo] <= a[mid]) {              // sol yarı a[lo..mid] sıralı
            if (a[lo] <= x && x < a[mid]) {
                hi = mid - 1;
            } else {
                lo = mid + 1;
            }
        } else {                            // sağ yarı a[mid..hi] sıralı
            if (a[mid] < x && x <= a[hi]) {
                lo = mid + 1;
            } else {
```

```
                hi = mid - 1;
            }
        }
    }
    return -1;
}
```

Python **python/H04_09_dondurulmus_dizi.py · satır 15–33**

```python
def dondurulmus_ara(a, x):
    global yoklama
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        yoklama += 1
        if a[mid] == x:
            return mid
        if a[lo] <= a[mid]:                 # sol yarı a[lo..mid] sıralı
            if a[lo] <= x < a[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                               # sağ yarı a[mid..hi] sıralı
            if a[mid] < x <= a[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1
```

Farklılaşan satırlar. Python zincirleme karşılaştırmaya izin verir: `a[lo] <= x < a[mid]` C/C++'taki `a[lo] <= x && x < a[mid]` ile, `a[lo] == a[mid] == a[hi]` ise `a[lo] == a[mid] && a[mid] == a[hi]` ile aynıdır. C'de `a == b == c` de derlenir ama anlamı farklıdır: önce `a == b` (0 ya da 1) hesaplanır, sonuç c ile karşılaştırılır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 108, 165; Python 61, 88, 136.)

A.10 Örnek 10 · Cevap üzerinde ikili arama: tamsayı karekök ve sayı tahmin oyunu (en çok $\lfloor \log_2 N \rfloor + 1$ soru)

Python **python/H04_10_karekok_tahmin.py · satır 18–37**

```python
def cok_buyuk_mu(r, n):
    return r > n // r


# ⌊√n⌋ = (P'nin doğru olduğu ilk r) - 1. Aranan bir dizi elemanı değil,
# [lo, hi] aralığındaki ilk D'dir. Değişmez: P(lo - 1) = Y, P(hi) = D.
def tamsayi_karekok(n):
    global adim
    if n < 2:
        return n
    lo = 1                                  # P(0) tanım gereği Y
    hi = min(n, 4294967296)                 # P(hi) = D
    while lo < hi:
        mid = (lo + hi) // 2
```

```
        adim += 1
        if cok_buyuk_mu(mid, n):
            hi = mid        # ilk D: mid ya da solu
        else:
            lo = mid + 1    # ilk D: mid'in sağı
    return lo - 1
```

Farklılaşan satırlar. C'de `r * r` 64 bitte taşabileceğinden yüklem `r > n / r` biçiminde yazılır; Python tamsayıları taşmaz ama aynı adımlar için aynı biçim korunur ve kâhin olarak `math.isqrt` kullanılır. Soru dağılımı hesabında C küçük bir (boyut, adet) dizisi ve `boyut_ekle` yardımcısı, C++ `std::map` ve yapılandırılmış bağlama (`auto& [boyut, adet]`), Python sözlük (`dict`) kullanır. (Gösterilen fonksiyonların dışındaki fark satırları: Python 70, 112.) Üç dil kesiti: C++ farkları: Bölüm 4.4.

A.11 Örnek 11 · İkiye bölme yöntemiyle denklem kökü: $x^3 - x - 2 = 0$; tolerans ve yineleme sayısı

C++ **cpp/H04_11_ikiye_bolme.cpp · satır 32–61**

```
static double ikiye_bolme(double a, double b, double tol, bool yaz)
{
    double fa = f(a);
    yineleme = 0;
    durma = "tolerans";
    while (b - a > tol && yineleme < EN_COK_YINELEME) {
        double m = a + (b - a) / 2;
        if (m <= a || m >= b) {         // a ile b komşu double: dur
            durma = "kayan nokta sınırı";
            break;
        }
        double fm = f(m);
        yineleme++;
        if (yaz) {
            std::printf("  %2d  %.9f  %.9f  %.9f  %10.3e\n", yineleme, a, b,
                        m, fm);
        }
        if (fm == 0.0) {                // double'da tam sıfır
            durma = "f(m) == 0";
            return m;
        }
        if ((fa < 0.0) == (fm < 0.0)) { // kök sağ yarıda
            a = m;
            fa = fm;
        } else {                        // kök sol yarıda
            b = m;
        }
    }
    return a + (b - a) / 2;
}
```

Python **python/H04_11_ikiye_bolme.py · satır 26–48**

```
def ikiye_bolme(a, b, tol, yaz):
    global yineleme, durma
    fa = f(a)
```

```
    yineleme = 0
    durma = "tolerans"
    while b - a > tol and yineleme < EN_COK_YINELEME:
        m = a + (b - a) / 2             # Python float = IEEE 754 double
        if m <= a or m >= b:            # a ile b komşu double: dur
            durma = "kayan nokta sınırı"
            break
        fm = f(m)
        yineleme += 1
        if yaz:
            print(f"  {yineleme:2d}  {a:.9f}  {b:.9f}  {m:.9f}  {fm:10.3e}")
        if fm == 0.0:                   # double'da tam sıfır
            durma = "f(m) == 0"
            return m
        if (fa < 0.0) == (fm < 0.0):    # kök sağ yarıda
            a = m
            fa = fm
        else:                           # kök sol yarıda
            b = m
    return a + (b - a) / 2
```

Farklılaşan satırlar. Python `float`'u C/C++ `double`'ı ile aynı IEEE 754 ikili64 biçimidir; işlemler aynı sırayla yapıldığından üç dilde bit bit aynı sonuç ve aynı çıktı elde edilir (`%.9f`, `%10.3e` ↔ `f"{x:.9f}"`, `f"{x:10.3e}"`). Python'da `/` her zaman kayan noktalı bölmedir. Durma nedeni C'de `const char *`, C++'ta `std::string`, Python'da `str` olarak tutulur. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 18.)

A.12 Örnek 12 · Genel arama: karşılaştırıcı işaretçisiyle struct dizisinde arama; bsearch / std::lower_bound / bisect

C++ **cpp/H04_12_genel_arama.cpp · satır 39–56**

```
template <class T, class Kucuk>
long genel_ara(const T& anahtar,
               const std::vector<T>& a, Kucuk kucuk)
{
    std::size_t lo = 0, hi = a.size();
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (kucuk(a[mid], anahtar)) {
            lo = mid + 1;   // anahtar > öge
        } else if (kucuk(anahtar, a[mid])) {
            hi = mid;       // anahtar < öge
        } else {
            return static_cast<long>(mid);  // eşdeğer
        }
    }
    return -1;
}
```

Python `python/H04_12_genel_arama.py · satır 27–40`

```python
def genel_ara(anahtar, a, key):
    global yoklama
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        k = key(a[mid])
        yoklama += 1
        if anahtar == k:
            return mid
        elif anahtar > k:
            lo = mid + 1    # anahtar > öge
        else:
            hi = mid        # anahtar < öge
    return -1
```

Farklılaşan satırlar. C'de genel arama `void *` taban, öge boyutu ve `int (*kars)(const void *, const void *)` karşılaştırıcısıyla yazılır; öge adresi `taban + i * boyut` bayttır. C++'ta şablon (`template`) türü ve yüklemi derleme zamanında bağlar; karşılaştırıcı bir "küçüktür" yüklemidir ve lambda olabilir. Python'da fonksiyonlar birinci sınıf değerdir: `key` fonksiyonu kayıttan anahtarı çıkarır (`bisect_left(..., key=...)` Python 3.10'da eklendi). (Gösterilen fonksiyonların dışındaki fark satırları: C++ 97, 153; Python 71, 115.) Üç dil kesiti: karşılaştırıcı sözleşmeleri: Bölüm 8.

A.13 Örnek 13 · Doğrusal ve ikili arama süre karşılaştırması ve kesişim noktası

C++ `cpp/H04_13_sure_karsilastirma.cpp · satır 24–48`

```cpp
static long dogrusal_ara(const std::vector<int>& a, int x)
{
    for (std::size_t i = 0; i < a.size(); i++) {
        if (a[i] == x) {
            return static_cast<long>(i);
        }
    }
    return -1;
}

static long ikili_ara(const std::vector<int>& a, int x)
{
    long lo = 0, hi = static_cast<long>(a.size()) - 1;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2;
        if (a[mid] == x) {
            return mid;
        } else if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return -1;
}
```

Python python/H04_13_sure_karsilastirma.py · satır 20–37

```python
def dogrusal_ara(a, x):
    for i in range(len(a)):
        if a[i] == x:
            return i
    return -1


def ikili_ara(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

Farklılaşan satırlar. Ölçülecek iş C'de fonksiyon işaretçisi + `void *` veri yapısıyla, C++'ta lambda (`[&] { … }`), Python'da `lambda:` ile `olc_ortanca`'ya verilir. Sayaçlı eşler sayacı C'de işaretçi parametreyle (`long *kars`), C++'ta başvuruyla (`long& kars`), Python'da demet dönüşüyle (`return i, kars`) iletir. Süreler yalnızca `--olc` ile ve stderr'e yazıldığından stdout üç dilde aynıdır. (Gösterilen fonksiyonların dışındaki fark satırları: C++ 51, 140; Python 46, 118.)

A.14 Örnek 14 · Satır ve sütunları sıralı matriste merdiven araması: en çok m + n − 1 adım

Python python/H04_14_merdiven_aramasi.py · satır 30–43

```python
def merdiven_ara(a, x):
    global adim
    m, n = len(a), len(a[0]) if a else 0
    r, c = 0, n - 1             # Python'da c = -1 sorun değil
    while r < m and c >= 0:
        v = a[r][c]
        adim += 1
        if v == x:
            return r, c
        elif v > x:
            c -= 1          # sütunu ele
        else:
            r += 1          # satırı ele
    return None
```

Farklılaşan satırlar. C'de matris satır öncelikli düz dizide tutulur (`a[r * n + c]`: AP-1 H10 adres formülü) ve bulunan konum düz indis olarak döner; C++'ta `std::vector<std::vector<int>>` ve `std::pair` konum, Python'da liste listesi ve `(r, c)` demeti kullanılır. C ve C++'ta `size_t` sütun indisi −1 olamayacağından `c` "sütun + 1" olarak tutulur; Python'da `c = n - 1` ile başlanıp `c >= 0` sınanır. Kâhin değer tablosu C'de `calloc` ile bayt dizisi, C++'ta `std::vector<char>`, Python'da `set`. (Gösterilen fonksiyonların dışındaki fark satırları: Python 104.) Üç dil kesiti: C++ farkları: Bölüm 4.6.